

Claude Orchestration Framework

Version 1.3.0 – consolidated printable render, 2026-08-24

github.com/abhinavsehgal/claude-orchestration-framework · MIT license

Generated from `docs/*.md` – the Markdown in the repo is the source of truth.

Claude Orchestration Framework 1.3.0

- 00 – Quickstart: onboard any project (Claude Code, many repos)
- 01 – Principles
- 02 – Architecture
- 03 – Agents Guide
- 04 – Handoff Schema
- 05 – Rules and Skills
- 06 – Invocation Modes
- 07 – Folder Structure
- 08 – Common Pitfalls
- 09 – Runbook: Bootstrap a New Project
- 10 – Hook-Based Hardening (Optional)
- 11 – Project Truth, Learnings, and the Evidence Ladder
- 12 – Multi-Repo Workspaces (web + mobile + microservices)
- 13 – Standing Routines (scheduled autonomy)

00 – Quickstart: onboard any project (Claude Code, many repos)

The whole framework as one step-by-step walk. Every step says **what to do**, **why**, **what to paste**, and **how you know it worked**. Written for someone who has never seen the framework; the long version of each part is linked. Based on v1.2.0 and the Claude Code docs as verified on 2026-08-22 – if a step here disagrees with the chapter it links to, the chapter wins.

Time: 15 min once per person · 2-4 h per repo · one afternoon for the workspace.

Part 0 • The words (read once)

Nine words. Everything else in this guide is made of these.

Word	Means
Orchestrator	The manager. Reads your task, decides who should do it, writes them a note, checks their work. Never touches code itself.
Specialist	A worker who does one kind of job (the API, the database, the UI, testing...). Touches only its own corner.
Handoff	The note the manager writes the worker: what to do, what the manager already knows (with proof), what would prove the manager wrong, what not to touch.
Return	The note the worker writes back: what it checked, what it changed, what it couldn't verify, what the manager must not pass on as fact.
Rule	A sticky note on a drawer. When Claude reads a file in that drawer, the note appears. Lives in <code>.claude/rules/</code> , scoped with <code>paths: .</code>
Skill	A recipe card for a job you do again and again (<code>/commit-push-pr</code> , the engineering playbook). Lives in <code>.claude/skills/</code> .
Hook	A tripwire. Runs a script when something happens (before an edit, when Claude tries to stop). Optional – add later.
Workspace	One desk with every repo's folder on it, plus one manager who only coordinates between repos. Its own small git repo.
Contract	The promise between a service and everyone who calls it (an API shape, an event payload, a shared package). The thing that breaks when two repos change at different times.

Part 1 • Before you start (15 minutes, once per person)

1. Make sure you have the four tools

Why: Claude Code runs the agents; git and jq are used by the workspace scripts; node runs hooks.

```
# Each of these should print a version, not an error
claude --version      # Claude Code 2.x or later
git --version
jq --version
node --version        # hooks are node scripts
```

Missing `claude` ? Install from the Claude Code docs (code.claude.com). Missing `jq` ? `brew install jq` on a Mac, `winget install jqlang.jq` on Windows.

✓ **You know it worked when:** all four print a version, and `claude` in any folder opens a session.

2. Get the framework onto your disk

Why: the framework is a folder of templates and two copy-paste prompts. You never install it into your project – you point Claude at it.

```
mkdir -p ~/frameworks
git clone https://github.com/abhinavseghal/claude-orchestration-framework.git
      ~/frameworks/claude
echo ~/frameworks/claude      # remember this path – you will paste it into the prompts
```

✓ `ls ~/frameworks/claude` shows `docs` `prompts` `templates`.

3. Read one file

Why: twenty minutes now saves a confused afternoon later. Skip the rest of the docs for now.

Open `docs/09-RUNBOOK.md`. That is the long version of Part 2 below.

✓ You can say in one sentence what the orchestrator does and does not do. (It coordinates. It never edits.)

Part 2 • One repo (2-4 hours – repeat for every repo)

Every repo gets its own manager and its own workers. Yes, every one – even a tiny service. The workspace in Part 3 only works if each repo already has this. Start with the repo you know best.

1. Open the repo on a fresh branch

Why: the setup creates new files. A clean branch means you can throw it all away if you don't like it.

```
cd ~/code/<your-repo>
git checkout main && git pull
git checkout -b setup/claude-orchestration
claude
```

✓ `git status` is clean; the session banner shows the repo path.

2. Run the INVENTORY prompt (look, don't touch)

Why: Claude scans the repo and *proposes* which specialists this repo needs. It writes nothing. You correct the proposal before anything is created.

Open `~/frameworks/claude/prompts/INVENTORY-PROMPT.md`, copy all of it, paste it into the session.

Replace `<framework path>` with the path from Part 1 step 2. Send.

It comes back with: a list of proposed specialists (usually 4-8), proposed rule files with their `paths:` globs, and a list of **Open Questions**. Answer the questions. Cross out specialists that feel wrong. Fewer

is better.

⚠ **The orchestrator must be named** `<repo-name>-orchestrator` (for example `orders-orchestrator`). Not just `orchestrator`. In Part 3 every repo’s manager is delegated to by name; names must be unique.

✓ You have a short list of specialist names you agree with, and `git status` is still clean.

3. Run the BOOTSTRAP prompt (now it builds)

Why: this creates all the files. It first **backs up** anything already in `CLAUDE.md` and `.claude/` to `.claude-pre-bootstrap-backup/` and asks before overwriting – so a `CLAUDE.md` your team already wrote is safe (you get a 3-pane diff before the merge).

In the **same session**, paste `~/frameworks/claude/prompts/BOOTSTRAP-PROMPT.md` (path replaced again). It shows you each file before saving. Say yes to each one you agree with.

What it creates:

File	What it is
<code>CLAUDE.md</code>	The router. Short (< 200 lines). Loaded into every session.
<code>.claude/agents/<repo>-orchestrator.md</code>	The manager.
<code>.claude/agents/<name>.md</code> ×N	The workers.
<code>.claude/rules/<area>.md</code>	The sticky notes, each with a <code>paths:</code> glob.
<code>.claude/skills/<repo>-engineering/SKILL.md</code>	The six-gate working method every task follows.
<code>.claude/skills/investigate-bug/</code> , <code>build-feature/</code>	Two starter recipe cards.
<code>docs/ai-context/PROJECT.md</code>	“What is true right now” – what’s live where, verified commands.
<code>docs/ai-context/LEARNINGS.md</code>	Decisions, failed approaches, corrections.
<code>docs/ai-context/HANDOFF_SCHEMA.md</code> , <code>INDEX.md</code> , <code>GLOSSARY.md</code>	The note format, the map, the one-name-per-thing list.
<code>docs/<AREA>-BACKLOG.md</code>	Where “we’ll do it later” must be written down.

✓ `ls .claude/agents` lists your orchestrator and specialists. Your normal build still passes.

4. Check the manager shows up

Why: an agent file with a typo in its `name:` silently doesn’t load.

Exit the session (`/exit`) and start a new one: `claude`. Type `/agents`.

✓ You see `<repo>-orchestrator` and every specialist. `/context` lists `CLAUDE.md` under **Memory files**.

△ Not there? The file must be in `.claude/agents/`, the `name:` line must match what you expect, and the YAML between the `---` lines must be valid. Fix, restart the session, look again.

5. Give it one real, small job

Why: the only proof the setup works is watching a handoff go out and a return come back.

```
claude --agent <repo>-orchestrator
```

Type a real bug or tiny feature from your tracker – one that needs maybe 30 minutes of human work.

Watch for three things: it restates the task; it writes a `handoff:` block (YAML) with `claims`, `failure_condition`, `in_scope`; the specialist answers with a `return:` block that lists `files_changed` and `tests_run`.

✓ You saw both blocks. The change is small and correct. If the specialist *refused* a vague handoff – that's the framework working, not failing.

6. Commit, open a PR, merge

Why: teammates get the agents the moment they pull. The backup folder stays local (gitignored).

```
git add CLAUDE.md .claude/ docs/ .gitignore
git commit -m "chore: bootstrap Claude Code orchestration framework"
git push -u origin setup/claude-orchestration
gh pr create --base main --title "Bootstrap Claude orchestration"
```

✓ PR merged. A teammate pulls, starts `claude`, types `/agents`, and sees the same list.

7. Later, not now: turn on hooks

Why: hooks are tripwires (block a stop while the build is red; force a correction to become a rule; refuse to end a turn after a production push with stale docs). Add them only after you've seen people skip the written rules. `docs/10-HOOK-HARDENING.md` is the recipe.

```
# when you're ready:
mkdir -p .claude/scripts
for f in ~/frameworks/claude/templates/hooks/*.mjs.template; do cp "$f"
    ".claude/scripts/${basename "$f"}.template"; done
# fill the <PLACEHOLDERS> in each script, then merge templates/hooks/settings.json.snippet
    into .claude/settings.json
```

△ `settings.json` is read when a session starts. After installing: `/exit`, start a new `claude`, then test. Stop hooks talk on **stderr**; a build killed by the cap is *inconclusive*, not failed.

Now do Part 2 again for the next repo. Two repos done with working orchestrators is the minimum before Part 3.

Part 3 • The workspace (one afternoon, once per team)

Only when tasks keep saying “change the orders service *and* the web app”. The workspace is a small extra repo that holds one coordinating manager, a map of who owns what, and the rules for changing contracts. It holds **no workers** – they stay in their repos. Long version: `docs/12-MULTI-REPO-WORKSPACES.md`.

Two Claude Code facts shape everything below (verified 2026-08-22): when you start `claude` in the workspace folder, a child repo’s `CLAUDE.md` and rules load as Claude reads its files – but a child’s **hooks and permission rules never load**, and a child’s agents are not visible. So the workspace never edits a child itself; it hands each job to the child’s own orchestrator in the child’s own session, where everything the child’s team built still applies.

1. Create an empty repo and clone it

Why: the workspace is its own repo so the map and the rules are versioned and shared. It has no CI and deploys nothing.

```
gh repo create <team>-workspace --private --clone
cd <team>-workspace
```

✓ You’re inside an empty folder with a `.git`.

2. Copy the workspace templates into place

Why: thirteen files, each with a fixed home. The bootstrap script does the copying AND fills the per-repo placeholders, `.gitignore`, the `settings.json` deny block and the service-map rows from `workspace.json` – so fill the manifest first (step 3) if you want the short path:

```
cp ~/frameworks/claude/templates/workspace/workspace.json.template workspace.json # fill
it, then:
bash ~/frameworks/claude/templates/workspace/bootstrap.sh.template ~/frameworks/claude
```

Or by hand (`templates/workspace/README.md` has the same table):


```
T=~/.frameworks/claude/templates/workspace
mkdir -p .claude/agents .claude/rules .claude/scripts docs/ai-context
cp $T/CLAUDE.md.template CLAUDE.md
cp $T/workspace.json.template workspace.json
cp $T/gitignore.template .gitignore
cp $T/orchestrator-agent.md.template .claude/agents/<team>-orchestrator.md
cp $T/contract-guardian-agent.md.template .claude/agents/contract-guardian.md
cp $T/service-mapper-agent.md.template .claude/agents/service-mapper.md
cp $T/cross-repo-contracts.md.template .claude/rules/cross-repo-contracts.md
cp $T/sync-repos.sh.template .claude/scripts/sync-repos.sh
cp $T/delegate.sh.template .claude/scripts/delegate.sh
cp $T/return-schema.json .claude/scripts/return-schema.json
cp $T/settings.json.snippet .claude/settings.json
cp $T/SERVICE_MAP.md.template docs/ai-context/SERVICE_MAP.md
cp $T/CONTRACTS.md.template docs/ai-context/CONTRACTS.md
cp ~/.frameworks/claude/templates/HANDOFF_SCHEMA.md.template docs/ai-context/HANDOFF_SCHEMA.md
chmod +x .claude/scripts/*.sh
```

✓ `find . -type f -not -path './.git/*'` matches the layout in chapter 12.

3. Fill in the blanks – mostly in one file

Why: `workspace.json` is the source of truth: which repos, where they live, what each one's manager is called, every contract between them, and how much a delegated session may do. The scripts read it; you never hard-code a path anywhere else.

Open `workspace.json`. For each repo fill `name`, `path` (e.g. `services/orders`), `url`, `default_branch`, `orchestrator` (exactly the name from Part 2 – `orders-orchestrator`), `build`, `test`. For each contract fill `producer`, `consumers`, `spec`, and a one-line `versioning` rule. Under `delegation`, keep `allowed_tools` narrow – it is the child's entire permission budget.

Then search every copied file for `<` and replace the remaining placeholders (`<WORKSPACE_SLUG>`, `<REPO_1_PATH>` ...). The `deny` block in `.claude/settings.json` must list every child path – that is what makes “the workspace never writes to a child” a fact, not a wish.

✓ `grep -rn "<[A-Z_0-9]*>" . --exclude-dir=.git` prints nothing. `jq . workspace.json` and `jq . .claude/settings.json` both print valid JSON.

4. Pull every repo onto the desk

Why: the children are plain clones in gitignored folders – disposable. The script clones what's missing and fetches what exists. It **never** switches or resets a branch; each repo's team owns that.

```
.claude/scripts/sync-repos.sh
```

✓ One `==` line per repo, no `⚠` warnings. A warning means that repo has no `<repo>-orchestrator` yet – go back to Part 2 for it.

5. Start the workspace session

Why: the only intended way to use the workspace is as its orchestrator.

```
claude --agent <team>-orchestrator
```

✓ `/agents` shows `<team>-orchestrator`, `contract-guardian`, `service-mapper` – and **not** the children's agents. That is correct: children are reached through `delegate.sh`, not spawned here.

6. Commit the workspace

```
git add -A
git commit -m "chore: bootstrap cross-repo workspace"
git push -u origin main
```

✓ `git status` shows no child repo files – the clones are ignored, the map and rules are committed.

7. The first cross-repo job – add something

Why: the workspace's whole job is ordering changes across a contract: the service that *produces* a field changes first, is deployed, and only then do the apps that *read* it change.

In the workspace session, ask for something additive that crosses one contract:

```
Add an `estimated_delivery` field to the orders API response and show it on the web order page.
```

Watch for: it reads `CONTRACTS.md`; it says "producer first: orders, then web"; it writes one handoff per repo to `.claude/handoffs/`, each with `repo:` and `contract_impact: additive`; it runs `.claude/scripts/delegate.sh orders <handoff>` – which starts `claude -p --agent orders-orchestrator` *inside* the orders repo, so that repo's rules and hooks apply – and reads the `return` from the JSON it prints.

✓ Two delegations, in that order. Each return lists `consumers_grepped`. `git status` in the workspace root is clean – the workspace itself changed nothing. `.claude/returns/` holds the two JSON results, each with a `session_id` you could `--resume`.

8. Break it on purpose – rename something

Why: a rename is a breaking change. The correct answer is a refusal.

```
Rename `total` to `grand_total` in the orders API response.
```

✓ It refuses to run it as one change and proposes three: add `grand_total` → move every consumer → remove `total`. If it just did the rename, the contract rule is not loading – check `.claude/rules/cross-repo-contracts.md` has no `paths:` (so it loads every session) and `/context` lists it.

Part 4 • Every day (the whole thing in one table)

You want to...	Where	Do this
Fix a typo, ask a question	That repo	Plain <code>claude</code> . The framework stays out of your way.
Do a medium/complex job inside one repo	That repo	<code>claude --agent <repo>-orchestrator</code> . Describe the job. Read the handoff and the return.
Work on two things at once without collisions	That repo	<code>claude --worktree</code> – an isolated checkout per session; never run two builds in one directory.
Ship what you did	Any	<code>/commit-push-pr</code> – it refuses to push to <code>main</code> , builds first, never stages secrets.
Change something in two or more repos	The workspace	<code>claude --agent <team>-orchestrator</code> . It orders the work producer → consumer and delegates one repo at a time.
Ask “is this API change safe?”	The workspace	Ask the orchestrator to run <code>contract-guardian</code> . It greps every consumer and answers with file:line, never an opinion.
Correct Claude (“no, use X”)	Any	With hooks: it is stopped and asked to draft a rule patch. Without: say “make that a rule”. Never accept “I’ll remember” – memory is machine-local.
Say “we’ll do that later”	Any	It must be written into <code>docs/<AREA>_BACKLOG.md</code> before the turn ends. Spoken follow-ups vanish.
Push to production	That repo	Same turn: update <code>CHANGELOG.md</code> , the affected <code>docs/ai-context/</code> map, and <code>PROJECT.md</code> “what is live where”. With hooks, the doc-freshness gate refuses to end the turn until you do.
Run a job from a script or CI	That repo	<code>claude -p "<task>" --agent <repo>-orchestrator --output-format json</code> (no <code>--bare</code> if you want the repo’s hooks and rules).

Part 5 • When it goes wrong

You see	It usually means	Fix
The agent isn't in <code>/agents</code>	Wrong folder or frontmatter	Must be in <code>.claude/agents/</code> ; <code>name:</code> must match; valid YAML between the <code>---</code> lines. Restart the session.
A rule never shows up	Its <code>paths:</code> glob doesn't match, or the file was never <i>read</i>	Run <code>find . -path "<the glob>"</code> . Rules load when Claude reads a matching file; a brand-new file written blind matches nothing – that is what the optional rule- surfacing hook covers.
From the workspace, the children's agents are missing	By design	Children are reached through <code>delegate.sh</code> . To spawn a child's agents in-session (investigation only), start with <code>claude --add-dir <child></code> .
<code>delegate.sh</code> exits with "handoff does not carry repo:"	The handoff file is missing its <code>repo:</code> line	Add <code>repo: <name></code> exactly as in <code>workspace.json</code> . A note for one repo must never run in another.
<code>delegate.sh</code> hangs or asks nothing and fails	<code>-p</code> never prompts; a tool wasn't pre-approved	Widen <code>delegation.allowed_tools</code> in <code>workspace.json</code> <i>one tool at a time</i> ; check the child's result JSON under <code>.claude/returns/</code> .
The workspace session tried to edit a child and was denied	The deny block did its job	Correct behaviour. Delegate instead.
A hook never fires	Installed mid-session, or wrote to the wrong channel	<code>/exit</code> and start a new session. Stop hooks must write to stderr and exit 2; PreToolUse hooks write to stdout.
The build-gate blocks on a "failed" build that was just slow	The cap is too low	Raise <code>BUILD_TIMEOUT_MS</code> ; a killed run must exit 0 (the v1.2 template does).
The specialist refused the task	The handoff was vague	That's correct behaviour. Add the missing field it named (usually <code>failure_condition</code> or evidence on a claim) and re-send.

You see	It usually means	Fix
Something worked last month and now doesn't	The platform moved	Re-run <code>prompts/REFINEMENT-PROMPT.md</code> ; its "platform drift" section re-checks the Claude Code docs. Two framework claims went stale in three months once already (Pitfall 18).

Part 6 • Optional: let it run while you sleep (after two weeks)

Once real tasks have flowed through the framework for a couple of weeks, add your first **standing routine** – a narrow agent job on a schedule that opens small PRs behind review gates. The full pattern (seven conventions, starter catalog, budgets): `docs/13-STANDING-ROUTINES.md` .

- **What:** copy `templates/routine.md.template` to `docs/routines/<name>.md` , fill the charter, schedule it – a product scheduler, or CI cron running headless `claude -p` inside the repo (the repo's hooks fire).
- **Why:** maintenance becomes a stream instead of a project. The public reference fleet merged 180 routine PRs in a few weeks at ~1-in-50 noise.
- **Start with ONE of:** doc-drift checker · dead-code remover · flaky-test root-causer (in a workspace: the contract-drift checker – Part 3's guardian on a clock).
- **The safety rules:** PRs only · never self-merge · per-run budget + attempt caps · a checked completion write · one reporting channel.
- **You know it worked when:** every run posts to its channel – including "nothing to do" – and the first wrong PR produces a tuning-log entry in the charter, not just a closed PR.

The checklist (print this)

Per person, once - [] `claude` , `git` , `jq` , `node` all print a version - [] Framework cloned to `~/frameworks/claude`

Per repo - [] Branch `setup/claude-orchestration` - [] INVENTORY prompt → specialist list agreed - [] BOOTSTRAP prompt → files created; orchestrator named `<repo>-orchestrator` - [] New session → `/agents` lists them; `/context` lists `CLAUDE.md` - [] One real small job via `claude --agent <repo>-orchestrator` → saw `handoff:` and `return:` - [] PR merged; a teammate sees the agents too - [] (later) hooks installed in `.claude/settings.json` , session restarted, tested

Per team, once – after ≥ 2 repos are done - [] Workspace repo created; 13 template files in place; no `<PLACEHOLDER>` left - [] `workspace.json` : every repo + every contract filled; `allowed_tools` narrow - [] `.claude/settings.json` deny block lists every child path - [] `.claude/scripts/sync-repos.sh` → no warnings - [] `claude --agent <team>-orchestrator` → `/agents` shows the three workspace agents only - [] Additive cross-repo job → producer first, two delegations, consumers grepped, workspace tree clean - [] Rename job → refused and split into add / move / remove

Cross-links

- <https://abhinavsehgal.github.io/claude-orchestration-framework/> (GitHub Pages) or `docs/00-QUICKSTART.html` offline – this guide plus the other two editions as one page with tabs (open in a browser; generated from the three `00-QUICKSTART.md` files on 2026-08-24).
- `docs/09-RUNBOOK.md` – the long version of Part 2.
- `docs/10-HOOK-HARDENING.md` – hooks (Part 2 step 7).
- `docs/12-MULTI-REPO-WORKSPACES.md` – the long version of Part 3, with the verified platform behaviour behind every step.
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` – why the backlog, `PROJECT.md` and “push = freshen docs” rules exist.
- `docs/06-INVOCATION-MODES.md` – `claude` vs `--agent` vs `-p` vs worktrees.
- `docs/08-COMMON-PITFALLS.md` – Part 5, in full.
- `docs/13-STANDING-ROUTINES.md` + `templates/routine.md.template` – Part 6, in full (scheduled autonomy).

01 – Principles

The seven principles this framework rests on. Every other doc, prompt, and template implements these.

1. Orchestrator-worker pattern, documented not coded

One coordinator agent reads tasks, classifies them, picks the right specialists, and aggregates findings. Specialists do focused work in isolated context windows.

The orchestration is **prescriptive documentation**, not runtime code. The “orchestrator” is a markdown file with instructions; the harness (Claude Code) provides isolation, tool restrictions, and turn caps. There is no orchestration service, no message queue, no central process.

This matters because: - It works with any LLM that supports subagents - It’s fully transparent – you can read every rule - It’s reversible – delete `.claude/` and you’re back to default - Hard runtime enforcement (hooks, `settings.deny`) can be added later as a separate hardening pass

2. Specialists run in isolated context

Each specialist agent runs in its own context window. It does not see the orchestrator’s full conversation. It receives a focused prompt (the handoff) and returns a focused result (the return).

This is the **single most important defense against context flooding**. A specialist investigating a database bug doesn’t need to know about your unrelated UI redesign in another open thread. Isolation makes each specialist’s reasoning sharp and their output auditable.

3. Universal evidence rule – “no evidence, no claim”

If a claim cannot be tied to a `file:line`, `table:column`, rule, doc anchor, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule appears verbatim in: - The handoff schema (both directions) - The orchestrator’s “outbound discipline” - Every specialist’s “incoming handoff validation”

It is the **core defense against cascading hallucinations** across hops. An orchestrator that hallucinates “the bug is in `foo.ts:42`” must cite that path; the specialist re-verifies before editing; if wrong, the specialist returns `status: claim_rejected` with the evidence of what it actually found.

4. Failure condition – articulate what would prove you wrong

Every outbound handoff includes a `failure_condition` field: one sentence stating what observable evidence would prove the orchestrator’s hypothesis or delegation premise wrong. If the specialist observes that condition, it stops, returns `claim_rejected`, and includes the triggering evidence.

This is the **inverse of verify_before_acting**. Where `verify_before_acting` says “check these before acting,” `failure_condition` says “if you observe this, STOP.” Together they bracket the work in falsifiable claims.

Forcing the orchestrator to articulate `failure_condition` is itself a thinking discipline – if you can’t name what would falsify your hypothesis, you don’t have a hypothesis, you have a guess.

5. Tools and turns are runtime-enforced; everything else is discipline

The Claude Code harness enforces: - `tools:` allowlists (specialist physically cannot use tools not listed) - `disallowedTools:` denylists (subtract from inheritance) - `maxTurns:` (hard cap on agent execution) - `Agent(name1, name2, ...)` allowlists (orchestrator can only spawn allowlisted specialists) - `mcpServers:` per-agent MCP server scoping - Subagent context isolation

Everything else (handoff schema fields, refusal behavior, evidence rule, hop limits, “rules read before edit”) is **documentation discipline** – relies on agents following their instructions. This is acceptable because: - Subagents are isolated, so a misbehaving agent’s damage is bounded - The orchestrator catches schema violations on receipt of a return - A poorly-documented expectation can be tightened into a hook later if needed

Don’t conflate the two layers. When you say “the orchestrator only delegates to project specialists,” that’s enforced at runtime via `Agent(name1, name2, ...)`. When you say “specialists refuse vague delegations,” that’s documented in their instructions and only as enforced as the agent’s compliance.

6. Three-tier documentation

Project documentation should split into three clearly-labeled tiers:

Tier	Location	Purpose	Read by
Orientation maps	<code>docs/ai-context/<topic>.md</code>	50-150 lines per area; gotchas; cross-links to canonical	Agents (per task routing)
Canonical references	<code>docs/<UPPERCASE>.md</code>	Architecture, API, schema, business – full detail	Humans + agents (when deeper)
Frozen archive	<code>docs/_archive/<date>/</code>	Sprint reports, post-mortems, migration logs, snapshots	Audits only – never linked from active docs

This separation makes drift impossible. Active docs live in the first two tiers. The archive is append-only and never referenced by agents/rules/active docs. When something is no longer authoritative, it moves to archive – it doesn’t sit in `docs/` confusingly mixed with truth.

7. Default Claude Code stays default

Do not make the orchestrator the project default. Setting `agent: "<orchestrator-name>"` in `.claude/settings.json` would force every session into orchestrator mode, which means: - No Edit/Write tools (every code change requires `Agent(specialist)` delegation – heavy friction for typos) - A `maxTurns` cap restrictive for interactive iterative work - Default Claude Code system prompt replaced entirely by the orchestrator's body prompt

The orchestrator is for **medium/complex tasks** – explicitly invoked via `claude --agent <orchestrator-name>`. Plain `claude` stays available for everyday work. Specialists can also be invoked directly via `claude --agent <specialist-name>` for narrow domain work.

Three everyday invocation modes coexist by design (plus headless `claude -p` and dynamic workflows for scripts and sweeps). See `06-INVOCATION-MODES.md`.

What these principles get you

A team can run dozens of parallel sessions on the same project without context collision. Specialists never inherit baggage from unrelated work. Orchestrator decisions are auditable (every claim cites evidence). Hallucinations don't compound across hops. New engineers see a clean folder layout that tells them where to look.

The cost is upfront discipline: you spend a day writing the agent contracts, rules, and orientation maps. The payback is permanent – every future task benefits.

02 – Architecture

The physical layout of files in a project that uses this framework.

The full picture

```
your-project/
├── CLAUDE.md                                ← project router (auto-loaded by Claude Code)
├── .claude/
│   ├── agents/                             ← orchestrator + N specialists
│   │   ├── <project>-orchestrator.md
│   │   ├── <specialist-1>.md
│   │   ├── <specialist-2>.md
│   │   └── ...
│   ├── rules/                             ← path-globbed invariants
│   │   ├── <domain-1>.md
│   │   ├── <domain-2>.md
│   │   └── ...
│   ├── skills/                             ← repeatable workflows
│   │   ├── <project>-engineering/SKILL.md ← (v1.2) the six-gate playbook
│   │   ├── <workflow-1>/SKILL.md
│   │   └── ...
│   ├── scripts/                           ← (optional, Chapter 10) hook scripts
│   ├── settings.json                      ← project-wide Claude Code settings (team-shared,
├── committed)                             ← personal permissions allowlist (gitignored)
│   └── settings.local.json
├── docs/
│   ├── ai-context/                       ← orientation maps (read by agents)
│   │   ├── INDEX.md                     ← task → docs router
│   │   ├── HANDOFF_SCHEMA.md            ← bidirectional handoff schema
│   │   ├── ORCHESTRATION_SPOONFEEDER.md ← human usage guide
│   │   ├── PROJECT.md                   ← (v1.2) current truth – what is live where
│   │   └── LEARNINGS.md                 ← (v1.2) decisions / failures / bug patterns /
├── corrections
│   ├── GLOSSARY.md                       ← (v1.2) one name per concept
│   ├── HOOKS.md                          ← (optional) what is enforced mechanically
│   ├── <area>-experience.md              ← per-domain orientation, 50–150 lines each
│   ├── ARCHITECTURE.md                   ← canonical refs (full detail)
│   ├── API.md
│   ├── <YOUR-OTHER-CANONICAL-DOCS>.md
│   ├── <AREA>_BACKLOG.md                 ← (v1.2) deferred work, one per feature area
│   └── _archive/                         ← frozen historical
│       ├── README.md                     ← archive convention
│       └── <YYYY-MM>/                    ← dated reports, post-mortems
└── <your application code>/
```

What lives where, and why

`CLAUDE.md` (root)

The router. Auto-loaded into every Claude Code session in this project. Should be **under 200 lines**.

Contains: - Identity (one sentence about the project) - Golden rules (security, deployment, etc.) - Mandatory workflow (numbered steps for every task) - Routing tables (task → doc, task → agent) - Cross-links to the spoonfeeder, INDEX, and canonical docs

Does NOT contain: - Long technical detail (lives in canonical docs) - Per-domain gotchas (lives in `.claude/rules/*.md`) - Sprint history or audit content (lives in `docs/_archive/`)

`.claude/agents/`

One markdown file per agent. Frontmatter declares `name`, `description`, `tools`, `maxTurns`, optionally `model` / `memory` /etc. Body declares the agent's contract: when to use, when not, required reading, I CAN, I CANNOT, definition of done, plus the handoff validation + return schema sections.

Always includes exactly one orchestrator. Specialists are added based on the project's natural domain boundaries (typically 4-10 for a medium-complexity codebase; sizing table in `03-AGENTS-GUIDE.md`).

See `03-AGENTS-GUIDE.md` for how to design agents.

`.claude/rules/`

Path-globbed invariants. Each rule file's frontmatter has `paths:` listing path globs – a native Claude Code field: the rule loads when Claude reads a matching file (rules without `paths:` load at launch). When an agent is about to edit a file matching one of those globs, it must have read the rule first. The rule contains "hard rules," "what not to do," and cross-links.

Example: `.claude/rules/database.md` might apply to `src/db/**/*.ts` and contain rules like "never run raw SQL in route handlers; use the repository layer."

Rules carry the **hard-won gotchas** of your codebase – things that broke production once and must not break again. They are short, scoped, and grow over time.

`.claude/skills/`

Each skill is a directory with a `SKILL.md` file inside. The skill is a **repeatable workflow** – a template for tasks that recur. Examples: - `investigate-bug/SKILL.md` – bug investigation steps - `build-feature/SKILL.md` – feature build steps - `qa-flow/SKILL.md` – pre-release QA pass - `compliance-review/SKILL.md` – review for regulatory risks

Skills are invoked by name when the orchestrator (or a human) decides this task fits a known pattern.

`docs/ai-context/`

Orientation maps. Each file is 50-150 lines and covers one area (e.g. `auth-and-sessions.md`, `payments.md`, `realtime-sync.md`). Format: - Top: 2-sentence orientation - Middle: key file paths + table

list + the 3-5 gotchas worth knowing - Bottom: cross-links to canonical refs

These are what agents read when deciding “what do I need to know about before touching code in it.”
Cheaper than reading 1000-line canonical docs.

Special files in `docs/ai-context/` : - `INDEX.md` – the master router (task → docs + rules + agent map) - `HANDOFF_SCHEMA.md` – the bidirectional handoff schema (this framework’s central artifact) - `ORCHESTRATION_SPOONFEEDER.md` – the human-facing usage guide - `PROJECT.md` , `LEARNINGS.md` , `GLOSSARY.md` – (v1.2) the project-truth set; see `11-PROJECT-TRUTH-AND-LEARNINGS.md` - `HOOKS.md` (optional) – what the installed hooks enforce (`10-HOOK-HARDENING.md`) - `MIGRATION_LEDGER.md` (optional) – tracks doc/structure migration progress - `LEGACY_BACKUP.md` (optional) – frozen pre-refactor snapshot

`docs/<UPPERCASE>.md`

Canonical references. Full detail. Examples: - `ARCHITECTURE.md` – system architecture - `API.md` – endpoint reference - `TECH_STACK.md` – what’s installed and why - `BUSINESS_DOCUMENT.md` – non-technical product/business notes - `CHANGELOG.md` – release log

These don’t change file layout per-task; they’re the encyclopedic source of truth.

`docs/_archive/`

Frozen historical material. Sprint reports, post-mortems, dated audits, migration logs. Two rules: 1. Active docs (`CLAUDE.md`, `docs/ai-context/*` , agents, rules) **never link into** `_archive/` . 2. The archive is append-only – date subdirs, never reorganize within.

The `docs/_archive/README.md` documents these conventions for future contributors.

Why this layout works

Predictability for Claude

Every agent knows where to find: - Its own contract: `.claude/agents/<self>.md` - The handoff schema: `docs/ai-context/HANDOFF_SCHEMA.md` - Routing: `docs/ai-context/INDEX.md` - Rules for paths it might edit: `.claude/rules/*.md` matched via `paths` - Canonical refs: `docs/<UPPERCASE>.md`

There is no ambiguity. New tasks follow the same path every time.

Predictability for humans

A new engineer joining the project sees: - `CLAUDE.md` at root – tells them how Claude Code is set up - `docs/` – where documentation lives - `.claude/` – Claude-specific config - Application code in its standard tech-stack location

They can ignore `.claude/` if they don’t use Claude Code. The framework adds value without imposing tax on humans who prefer their own tooling.

Minimal coupling to tech stack

Nothing in `.claude/` or `docs/ai-context/` cares about your framework, language, or deploy target. The agents reference paths in your actual codebase, but the agent FILES themselves are pure markdown. You can reorganize your codebase entirely and only need to update the rule globs and the orientation maps' cross-links.

What does NOT belong in this framework

- **Application code.** Stays where your tech stack puts it.
- **Build config.** Stays at root (your `package.json`, `pyproject.toml`, `Cargo.toml`, `go.mod`, etc.).
- **CI workflows.** Stays in `.github/`, `.gitlab-ci.yml`, etc.
- **Test files.** Stays in your tests directory.
- **Generated artifacts.** Gitignored.

The framework is **purely additive** to whatever else lives in your repo.

Variations

Monorepos

For monorepos, you can either: - Have one `.claude/` at the root that knows about all packages, or - Have one `.claude/` per package + a root `.claude/` that orchestrates across packages

The first is simpler. The second is more isolated. Pick based on whether tasks routinely cross package boundaries. Two platform facts decide what goes where (verified 2026-08-22): per-package `CLAUDE.md` and `.claude/rules/` load **on demand** when Claude reads files in that package, but `.claude/settings.json` (hooks, permission rules) loads **only from the directory you start in**, and a package's `.claude/agents/` are visible only to sessions started in that package (or added with `--add-dir`). So gates live with the code they gate, and a root session does not inherit a package's hooks or agents. The official *monorepos and large codebases* page covers `claudeMdExcludes`, sparse worktrees and per-package skills.

Multiple repositories (web + mobile + microservices)

When the units are separate repos rather than packages, see `12-MULTI-REPO-WORKSPACES.md`: every repo keeps its own install, shared specialists become a plugin, and a workspace repo holds only the cross-repo orchestrator, the service map and the contract rules.

Mobile + web split

If your project has separate mobile and web codebases in one repo, your orchestrator typically delegates to a `mobile-ui` specialist OR a `web-ui` specialist depending on the task. They can share a `backend-api` specialist for the API-side work.

Multi-tenancy / multi-product

If you serve multiple products from one codebase, consider per-product orientation maps and per-product rules. The orchestrator can be one or per-product depending on whether tasks routinely cross products.

Heavy infra / DevOps focus

Add specialists for `infrastructure`, `observability`, `release-automation`. Their rule files cover Terraform/CDK/Kubernetes/Helm conventions.

ML / data heavy

Add specialists for `data-pipeline`, `model-training`, `inference-serving`. Their orientation maps cover dataset locations, experiment tracking, model versioning.

The architecture flexes; the principles don't.

03 – Agents Guide

How to design the orchestrator and specialists for your project.

The orchestrator

Every project has exactly **one** orchestrator. Naming convention: `<project-name>-orchestrator` (e.g. `acme-orchestrator`, `payments-orchestrator`).

Responsibilities

- Read incoming task. Restate it.
- Identify affected roles / domains.
- Read minimum docs (INDEX.md → 1-3 orientation maps).
- Pick the right specialists (typically 1-3 per task).
- Issue structured handoffs with claims + evidence + failure_condition.
- Receive returns. Verify rejected_claims. Re-verify unverified_claims before propagating.
- Aggregate findings into a final summary with the project's "Definition of Done."

Constraints

- **No Edit/Write tools.** The orchestrator coordinates; specialists implement.
- **Restricted Agent allowlist.** Only project specialists, not built-ins or plugins.
- **Modest maxTurns.** 20-30 turns is typical (read INDEX → delegate 1-3 times → aggregate).

Frontmatter template

```
---
name: <project>-orchestrator
description: Main task dispatcher for <Project>. Use for medium/complex tasks. Picks minimum
  docs/agents/skills; never loads everything.
tools: Read, Grep, Glob, Bash, WebFetch, WebSearch, TodoWrite, Agent(<specialist1>,
  <specialist2>, <specialist3>, ...)
maxTurns: 25
---
```

Specialists

A specialist owns one domain. Specialists are domain-shaped, not technology-shaped. Bad: `react-specialist`, `typescript-specialist`. Good: `frontend-ui`, `payments`, `realtime-sync`, `compliance`.

How many do you need?

Project size	Specialist count
Solo project / prototype	0-2 (just use plain <code>claude</code>)
Small team / single product	4-6
Medium codebase / multiple roles	6-10
Large codebase / multi-product	10-15

More than 15 is a smell – you’re probably making them too narrow.

Common specialists by project type

Web app (any stack)

Specialist	Domain
<code>frontend-ui</code>	UI components, routing, state management, client-side interactions
<code>backend-api</code>	API routes, server logic, request handling
<code>database</code>	Schema, migrations, query layer, indexes
<code>auth-security</code>	Authentication, authorization, sessions, secrets
<code>qa-functional</code>	Browser/E2E testing, regression checks
<code>release-devops</code>	Build, deploy, env config, observability

Mobile app

Specialist	Domain
<code>mobile-ui</code>	Screens, navigation, gestures, platform-specific UX
<code>mobile-state</code>	State management, persistence, offline sync
<code>mobile-native</code>	Platform bridges (iOS/Android native code or Flutter plugins)
<code>backend-api</code>	Server-side endpoints the app calls
<code>qa-mobile</code>	Device testing, simulator/emulator flows

Backend / API service

Specialist	Domain
api-routes	HTTP/gRPC route handlers
domain-logic	Core business logic
data-layer	DB access, repositories, ORM patterns
integrations	Third-party API clients
observability	Logging, metrics, tracing
release-devops	Deploy, env, secrets

ML / data project

Specialist	Domain
data-pipeline	ETL, ingestion, transformation
model-training	Training scripts, hyperparam tuning, experiment tracking
inference-serving	Model serving, latency, batching
data-quality	Validation, anomaly detection, monitoring

Cross-cutting (add to most projects)

Specialist	Domain
product-flow	Acceptance criteria, cross-role behavior, requirement clarification (LIGHT EDITS ONLY)
qa-functional	Tests across the system
security-privacy	Auth / permissions / sensitive-data review (REVIEW-ONLY)
legal-compliance	Regulatory review for relevant laws (REVIEW-ONLY)
release-devops	Build/deploy/cron/env
context-librarian	Maintains <code>.claude/</code> , <code>docs/ai-context/</code> , archives drift
docs-writer	Canonical doc updates

REVIEW-ONLY specialists

Some specialists should explicitly **not** edit code. Tag them in the description, give them no Edit/Write tools, and document the constraint in their I CANNOT section.

Examples: - `legal-compliance` – flags risks; never claims final legal advice - `security-privacy` – identifies vulnerabilities; doesn't ship fixes - `architecture-review` – reviews proposed designs;

doesn't implement

Frontmatter for REVIEW-ONLY:

```
---
name: legal-compliance
description: REVIEW-ONLY. Flags <list of regulations> risks. Produces attorney-checklist
            content. Never claims final legal advice.
tools: Read, Grep, Glob, WebFetch, WebSearch, TodoWrite
maxTurns: 12
---
```

Notably absent: `Edit`, `Write`, `Bash`. The harness physically prevents this agent from modifying files.

⚠ **Do NOT enable `memory` on REVIEW-ONLY agents.** Per Claude Code docs, enabling `memory` automatically grants Read/Write/Edit so the agent can manage its memory files – silently bypassing your REVIEW-ONLY contract.

Implementation specialists

Get full Edit/Write/Bash access plus relevant MCP tools.

Frontmatter for an implementation specialist:

```
---
name: backend-api
description: Builds and fixes API routes, server logic, request handling. Coordinates with
            database and auth-security specialists.
tools: Read, Edit, Write, Grep, Glob, Bash, TodoWrite
maxTurns: 20
---
```

For specialists that need MCP server access (e.g. browser testing), use the documented wildcard:

```
tools: Read, Edit, Write, Grep, Glob, Bash, TodoWrite, mcp__chrome-devtools__*,
        mcp__playwright__*
```

The wildcard is supported and covers all current + future tools from that MCP server.

Agent body structure (every agent)

Every agent file body should include these sections in order:

```

# <Agent Display Name>

<2-sentence description of the agent's domain.>

## When to use

- Bullet list of task types this agent owns

## When NOT to use

- Bullet list of task types that belong to other agents

## Required reading

1. `docs/ai-context/INDEX.md` – task → docs map
2. <area-specific orientation map>
3. **`.claude/rules/<your-rule>.md`** – always (when applicable)
4. <other rules / canonical refs>

## Incoming handoff validation

<paste the standard incoming handoff validation block – see template>

## Return schema (required)

<paste the standard return schema block – see template>

## I CAN

- Bullet list of what this agent is allowed to do

## I CANNOT

- Bullet list of what this agent must refuse
- Always include: "Push to main" (or your equivalent protected branch)

## Definition of Done

1. Files changed (paths)
2. Rule files read before editing
3. Tests run
4. Risks remaining
5. (any agent-specific Definition of Done items)

## Cross-links

- `<related rule>`
- `<related orientation map>`
- `<canonical doc>`

```

The “Incoming handoff validation” and “Return schema” sections are **identical across all specialists** – they enforce the universal handoff contract. See `templates/specialist-agent.md.template`.

Naming conventions

- Lowercase with hyphens: `backend-api`, `frontend-ui`, `legal-compliance`

- Filename matches `name : backend-api → backend-api.md`
- Be specific but not over-narrow: `payments` good, `stripe-webhook-handler` too narrow
- Cross-cutting concerns get their own agent: `qa-functional`, `security-privacy`, not folded into another specialist

When to add a new specialist vs. extend an existing one

Add a new specialist when: - The domain has its own gotchas (would warrant its own rule file) - Tasks in that domain frequently arrive - The existing specialists' "I CANNOT" sections would all reject this work

Extend an existing specialist when: - The work overlaps with an existing specialist's domain - The new domain is small (fits in 1-2 paragraphs of the existing agent's body) - You haven't seen 3+ tasks in this domain yet

When in doubt: don't add. It's easier to split a specialist later than to delete one that accumulated cruft.

Anti-patterns

✗ **Technology-named specialists.** `typescript-specialist` doesn't have a clear domain – what does it own that `frontend-ui` and `backend-api` don't already cover?

✗ **Layer-named specialists.** `database-specialist` is fine if "database" means a domain (schema, migrations, repository layer); not fine if it means "anyone touching SQL anywhere should consult me."

✗ **Specialists with overlapping `tools` and overlapping descriptions.** Confuses the orchestrator; it won't know which to pick.

✗ **Specialists with no `tools: field`.** They inherit everything, defeating defense-in-depth. Always declare `tools` explicitly.

✗ **Specialists that delegate to other specialists.** Subagents *can* spawn subagents (up to a configurable depth – see Pitfall 18), but a specialist that does so has become an un-audited orchestrator: the handoff schema, the evidence rule and the orchestrator's return-validation are all bypassed one level down. By framework convention a specialist returns to the orchestrator with `recommended_next_agent` and lets the orchestrator chain the work. The only sanctioned nesting is orchestrator → orchestrator (Chapter 12, multi-repo), where both hops carry the full schema.

✗ **Specialists whose "I CAN" includes "approve production deploys."** That's the project owner's call, not an agent's.

04 – Handoff Schema

The bidirectional schema for orchestrator ↔ specialist communication. This is the framework's central artifact.

Why a schema

Without structure, handoffs are free-form prompts. Specialists guess what's expected. Orchestrators don't commit to specific claims. Cascading hallucinations compound across hops because nothing forces evidence-to-claim binding at the boundary.

The schema closes both gaps with two YAML blocks (one per direction) and one universal evidence rule.

Universal evidence rule

No evidence, no claim. If a claim cannot be tied to a file, line, table, column, rule, doc, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule applies to both directions of every handoff. It is repeated verbatim in the orchestrator's outbound-discipline section and in every specialist's incoming-handoff-validation section.

Outbound: orchestrator → specialist

Every Agent-tool delegation from the orchestrator **MUST** include this YAML block at the top of the prompt.

```
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <orchestrator-name>
  to_agent: <specialist-name>
  goal: <one sentence – what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>, ...]
  claims:
    - claim: <factual statement the orchestrator commits to>
      evidence: <path/to/file.ts:42 OR rule:.claude/rules/foo.md OR
        doc:docs/ARCHITECTURE.md#section OR table:column OR test:path OR command:output>
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence – observable evidence that proves the orchestrator's
    hypothesis or delegation premise is wrong. If the specialist observes this, it MUST
    stop and return status: claim_rejected (or needs_clarification) instead of editing on
    a false premise.>
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch – even if "while we're in there">
  rules_to_read:
    - <.claude/rules/*.md path the specialist must read first>
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line – why the orchestrator is delegating>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line – what came back, or "n/a">
```

Field requirements (outbound)

Field	Required	Notes
<code>schema_version</code>	yes	Currently <code>1</code> . Bump on breaking changes.
<code>handoff_id</code>	yes	Short unique id. Specialist echoes it on return so they pair.
<code>from_agent</code>	yes	Always the orchestrator's name.
<code>to_agent</code>	yes	The specialist's <code>name</code> field.
<code>goal</code>	yes	One sentence with measurable outcome. "Fix" or "improve" without an outcome is too vague.
<code>task_class</code>	yes	One of the six values.
<code>affected_roles</code>	yes	Non-empty list.
<code>claims</code>	yes (may be empty list <code>[]</code>)	Every entry MUST have <code>evidence</code> . Use <code>confidence: low</code> for assumptions.
<code>verify_before_acting</code>	yes if task touches code	Empty allowed only for pure-research handoffs.
<code>failure_condition</code>	yes	One sentence. The inverse of <code>verify_before_acting</code> . Use <code>failure_condition: open-ended exploration; no specific premise to invalidate</code> only when no premise truly exists.
<code>in_scope</code>	yes	Non-empty list.
<code>out_of_scope</code>	yes	Empty list <code>[]</code> is valid. Missing key is not.
<code>rules_to_read</code>	yes	Use <code>["specialist will discover from path globs"]</code> if you don't know which rules apply.
<code>expected_artifacts</code>	yes	Drives the specialist's <code>files_changed</code> / <code>tests_run</code> later.
<code>hop_context</code>	yes	All three sub-fields required. Use <code>none</code> / <code>n/a</code> for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block.

```
return:
  schema_version: 1
  handoff_id: <copy from inbound – pairs the response>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why – out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, rule, doc anchor, or command output>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist-name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding the orchestrator must NOT propagate as fact>
```


Field requirements (inbound)

Field	Required	Notes
<code>schema_version</code>	yes	Must match outbound.
<code>handoff_id</code>	yes	Must echo inbound id verbatim.
<code>from_agent</code> / <code>to_agent</code>	yes	Self-explanatory.
<code>status</code>	yes	<code>completed</code> only if all <code>verify_before_acting</code> items checked. Use <code>claim_rejected</code> when an orchestrator claim was wrong.
<code>verified_claims</code>	yes (may be <code>[]</code>)	Empty list is valid. Missing key is not.
<code>rejected_claims</code>	yes (may be <code>[]</code>)	Empty list is valid. Non-empty implies <code>status: claim_rejected</code> or <code>blocked</code> .
<code>unverified_claims</code>	yes (may be <code>[]</code>)	Anything you accepted without re-checking goes here.
<code>evidence_used</code>	yes	Real paths/commands. "I read the codebase" is not evidence.
<code>files_changed</code>	yes (may be <code>[]</code>)	Empty list valid for review/audit handoffs.
<code>tests_run</code>	yes (may be <code>[]</code>)	Empty list valid for pure-doc changes.
<code>risks</code>	yes	Use <code>["none"]</code> if there are genuinely none.
<code>recommended_next_agent</code>	yes	<code>agent: "none"</code> if work is complete.
<code>do_not_pass_downstream_without_verification</code>	yes (may be <code>[]</code>)	Cascading-hallucination defense. Be liberal.

Refusal

If the inbound handoff fails the field requirements, the specialist responds with ONLY this block, does no other work, and waits for re-issue:

```
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong – missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Failure-condition observation rule

If at any point during the work the specialist observes the inbound `failure_condition`, it STOPS immediately and responds with the return schema using `status: claim_rejected` (or `needs_clarification` if ambiguous). It cites the triggering evidence in `evidence_used`. It does NOT push through a falsified premise – even if mid-edit, partial work has already been done, or the fix seems easy.

Worked examples (any tech stack)

Example 1 – Backend bug investigation

Outbound (orchestrator → backend-api):

```

handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: acme-orchestrator
  to_agent: backend-api
  goal: Find why /api/orders returns 500 for orders with > 100 line items.
  task_class: bug
  affected_roles: [customer]
  claims:
    - claim: The 500 originates in the order serialization step.
      evidence: src/api/orders/handler.ts:88
      confidence: high
    - claim: Order serialization runs synchronously and may exceed the function timeout.
      evidence: rule:.claude/rules/api.md
      confidence: medium
  verify_before_acting:
    - That src/api/orders/handler.ts:88 is still the serialization line.
    - That the timeout is actually exceeded (logs / metric, not just suspected).
  failure_condition: If logs show the 500 happens in the database query rather than
    serialization, STOP – the orchestrator's hypothesis about the failure location is
    wrong.
  in_scope:
    - src/api/orders/handler.ts
    - src/api/orders/serializer.ts
  out_of_scope:
    - The order schema (database concern; separate specialist if migration needed).
  rules_to_read:
    - .claude/rules/api.md
    - .claude/rules/observability.md
  expected_artifacts:
    - Root cause statement (file:line)
    - Smallest fix or "no bug – load test issue" verdict
  hop_context:
    parent_task: Customer reports cart > 100 items always errors at checkout.
    previous_specialist: none
    previous_findings_summary: n/a

```

Return (backend-api → orchestrator):

```

return:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: backend-api
  to_agent: acme-orchestrator
  status: claim_rejected
  verified_claims:
    - claim: src/api/orders/handler.ts:88 is the serialization line.
      evidence: src/api/orders/handler.ts:88 (confirmed via grep + read)
  rejected_claims:
    - claim: The 500 originates in serialization.
      evidence: Stack trace from production logs shows the failure is in the database query
        at src/db/repositories/orderRepo.ts:142, not serialization. The query loads ALL line
        items in a single SELECT without pagination, exceeding the 30s function timeout for
        orders with > ~140 line items.
  unverified_claims: []
  evidence_used:
    - src/api/orders/handler.ts:80-95
    - src/db/repositories/orderRepo.ts:140-150
    - command:grep -rn "loadOrderLineItems" src/
    - production logs (sampled 3 failing requests)
  files_changed: []
  tests_run: []
  risks:
    - Real fix requires pagination of the query. Out of this handoff's scope.
  recommended_next_agent:
    agent: database
    why: Pagination of orderRepo.loadOrderLineItems is the correct fix.
  do_not_pass_downstream_without_verification:
    - That the timeout value is actually 30s (read from env or config; I inferred from
      observed behavior, did not check).

```

Example 2 – Mobile feature build

Outbound (orchestrator → mobile-ui):

```

handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-feat-002
  from_agent: acme-orchestrator
  to_agent: mobile-ui
  goal: Add a pull-to-refresh gesture on the OrderHistoryScreen.
  task_class: feature
  affected_roles: [customer]
  claims:
    - claim: OrderHistoryScreen lives at src/screens/OrderHistoryScreen.tsx.
      evidence: src/screens/OrderHistoryScreen.tsx
      confidence: high
    - claim: The app uses React Native's RefreshControl for pull-to-refresh elsewhere.
      evidence: rule:.claude/rules/mobile-ui.md
      confidence: high
  verify_before_acting:
    - That OrderHistoryScreen uses a ScrollView or FlatList that can host RefreshControl.
  failure_condition: If OrderHistoryScreen renders a non-scrollable view (e.g. a Map or a
    custom virtualized list), STOP – the standard RefreshControl pattern won't apply and
    a different solution is needed.
  in_scope:
    - src/screens/OrderHistoryScreen.tsx
    - src/screens/__tests__/OrderHistoryScreen.test.tsx (add test)
  out_of_scope:
    - The order data fetching logic (already exists; just call refetch).
  rules_to_read:
    - .claude/rules/mobile-ui.md
  expected_artifacts:
    - Edit to OrderHistoryScreen.tsx adding RefreshControl
    - New test asserting onRefresh triggers refetch
  hop_context:
    parent_task: Product wants pull-to-refresh on order history per ACME-1234.
    previous_specialist: none
    previous_findings_summary: n/a

```

Example 3 – Compliance review (REVIEW-ONLY specialist)

Outbound (orchestrator → compliance):

```

handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-rev-003
  from_agent: acme-orchestrator
  to_agent: compliance
  goal: Review the new email opt-in flow for GDPR/CASL/CAN-SPAM compliance before launch.
  task_class: review
  affected_roles: [customer]
  claims:
    - claim: The opt-in flow records consent timestamp + IP + opt-in source.
      evidence: src/db/migrations/2026-05-01-add-consent-log.sql
      confidence: high
    - claim: Marketing emails currently send without an explicit unsubscribe link.
      evidence: src/email/templates/marketing.tsx:42
      confidence: medium
  verify_before_acting:
    - That the migration has been applied to staging.
    - That the consent log columns are actually written by the new opt-in handler.
  failure_condition: If the consent log table exists but the opt-in handler does NOT write to
    it, STOP – the consent capture is non-functional and the compliance review is moot.
  in_scope:
    - REVIEW-ONLY. No code edits.
    - src/email/**
    - src/db/migrations/2026-05-01-*
  out_of_scope:
    - Any code changes (escalate to email-templates or backend-api specialists).
  rules_to_read:
    - .claude/rules/security-privacy.md
  expected_artifacts:
    - Severity-ranked findings (blocker / high / medium / low)
    - Attorney-checklist of questions for counsel
  hop_context:
    parent_task: Pre-launch gate for new email opt-in feature.
    previous_specialist: none
    previous_findings_summary: n/a

```

Optional fields added in v1.2.0 (additive – `schema_version` stays 1)

Outbound, for multi-repo workspaces (Chapter 12) – omit in single-repo projects:

```

repo: <repo name from workspace.json – the ONLY repo this handoff may edit>
contract_impact:
  level: <none | additive | breaking>
  contracts: [<names from CONTRACTS.md>]
  consumers_to_update: [<repo names – required when level != none>]

```

Outbound, any project – the evidence-confidence class from Chapter 11 may be used in place of the three-value `confidence:` on a claim (`verified-code` / `verified-schema` / `verified-test` / `verified-git` / `documented-unverified` / `historical` / `unknown`). Specialists treat anything that is not `verified-*` exactly as they treat `confidence: low` .

Inbound, for multi-repo workspaces:

```
contracts_changed:
- contract: <name>
  change: <one line>
  backward_compatible: <true | false>
  consumers_grepped: [<repo>:<path>, ...]
```

Inbound, any project – `deferred_work:` lists anything the specialist is *not* doing that someone must (each item with the backlog file it was appended to). A return that names deferred work without a backlog path is incomplete (Chapter 11).

Versioning

- `schema_version: 1` is current.
- Additive changes (new optional fields) keep version 1.
- Breaking changes (renaming fields, changing field semantics) bump the version.
- Bumps require updating `HANDOFF_SCHEMA.md`, the orchestrator, and every specialist file in the same PR.

Why this schema is enough (without runtime hooks)

The schema is enforced by: 1. The orchestrator's instructions to ALWAYS emit the outbound block. 2. Every specialist's instructions to refuse delegations missing the block. 3. The orchestrator's instructions to consume the return block before merging work.

This is documentation enforcement – agents follow the rules because their instructions tell them to. It works because: - Subagents are isolated, so a misbehaving agent's damage is bounded - The orchestrator catches return-block violations (missing fields, wrong types) - Refusal is a one-shot rejection: if the specialist returns the refusal template, the orchestrator must re-issue with the missing fields – no work happens until the schema is satisfied

If you later want hard runtime enforcement, add a `PreToolUse` hook on the `Agent` tool that validates the outbound YAML block before letting the call proceed. That's a one-day project; the framework deliberately ships **no hooks in the default install** (the optional hook templates in Chapter 10 cover other gaps, not schema validation) because the document-only enforcement covers 95% of the value at 5% of the complexity.

05 – Rules and Skills

Two patterns for capturing knowledge that doesn't belong in agents: **rules** (path-globbed invariants) and **skills** (repeatable workflows).

Rules

A rule is a markdown file in `.claude/rules/` with frontmatter declaring which file paths it applies to. When an agent is about to edit a matching file, it must read the rule first.

When to write a rule

Write a rule when: - A bug shipped to production once and must not recur - A pattern is non-obvious from reading the code (e.g. "this column looks like a string but it's a JSON-encoded string in legacy rows") - A constraint crosses multiple files but isn't enforced by types/tests - An anti-pattern is tempting but wrong (e.g. "this looks like dead code but is actually called via reflection")

Don't write a rule for: - Things obvious from the code (well-named identifiers do that work) - Style preferences (lint config does that work) - One-off decisions (PR descriptions do that work) - Anything covered by an existing rule (extend the existing one)

Rule file structure

v1.2.0 – paths: is native. Claude Code loads `.claude/rules/*.md` itself: a rule with a `paths:` frontmatter list loads when Claude *reads* a file matching one of the globs; a rule without `paths:` loads at launch, with the same priority as `CLAUDE.md`; nested `.claude/rules/` in subdirectories load on demand; symlinked rule files work. v1.0 used a framework-private `applies_to:` field plus a hook. The field is now `paths:` so the platform and the optional rule-surfacing hook (Chapter 10, Pattern 1) read one source. The hook still earns its place for one case the native loader misses: a brand-new file that is *written* without ever being *read*.


```

---
name: <domain-name>
description: Scoped rules for <domain>. Read before editing any of the path globs below.
paths:
  - "<glob1>"
  - "<glob2>"
---

# <Domain Name> Rules

## Hard rules

### <Rule title – short imperative>

**Why:** <one-sentence reason – usually a past incident or invariant>

**How to apply:** <2–4 sentences – what to do, what helper to use, what to check>

### <Next rule>

...

## What NOT to do

- Bullet list of anti-patterns

## How to use this file

1. If you're editing any path in `paths`, read this file first.
2. Report which rule files were read in your task summary.

## Cross-links

- `<related orientation map>`
- `<related canonical doc>`
- `<related rule>`

```

Glob patterns

Use standard shell globs. Examples:

```

paths:
  - "src/api/**/*.ts"           # all TypeScript under src/api/
  - "src/db/migrations/**/*.sql" # all migration SQL files
  - "src/{lib,utils}/auth-*.ts"  # auth-related lib/util files
  - "components/payment/**/*.{tsx,jsx}" # payment components
  - "ios/Runner/**/*.swift"      # iOS Swift files
  - "lib/screens/**/*.dart"      # Flutter screens
  - "internal/handlers/**/*.go"   # Go API handlers
  - "app/models/*.rb"            # Rails models

```

Globs are matched against the relative path from project root.

Rule examples by tech

Backend / API

Always validate request body before reading database

****Why:**** Three production incidents traced to unvalidated body data flowing to SQL parameters.

****How to apply:**** Use `validateRequest(schema)` middleware in `src/lib/validation.ts`. Never read `req.body.<field>` directly in a handler.

Database / migrations

Migration order is sacred – never reorder, never edit applied migrations

****Why:**** Migrations apply in filename order. Editing an applied migration silently diverges staging from production.

****How to apply:**** New migrations get the next sequential timestamp prefix. Bug fixes to applied migrations require a NEW migration that supersedes the old behavior.

Frontend / UI

Never trust component props for security checks – re-validate on the server

****Why:**** A user can edit any prop via React DevTools.

****How to apply:**** Server-side handlers re-validate authorization. Props are for rendering, not authorization.

Mobile

iOS auto-zooms on input focus when font-size < 16px

****Why:**** iOS Safari WebView feature; jarring UX.

****How to apply:**** Set `font-size: 16px` minimum on all `<input>`, `<select>`, `<textarea>`. For visual size adjustment, scale via padding/transform, not font-size.

Infra / DevOps

Never commit secrets – use env vars + secrets manager

****Why:**** Once committed, secrets are in git history forever; rotation required.

****How to apply:**** Add new secrets via `<your-secrets-manager>`. The deploy pipeline injects them. Never `git add` files matching `.env*`.

Anti-patterns in rules

- ✗ **Long rationale paragraphs.** Rules should be ≤ 4 sentences per rule. Move long stories to the canonical doc.
- ✗ **Aspirational rules.** “We should always...” – if it’s not enforced, it’s not a rule. Either write it as enforced (with the helper / check) or move to a style guide.
- ✗ **Rules that contradict other rules.** Specialist confusion. Have the context-librarian agent reconcile.
- ✗ **Path-specific rules with no paths.** A rule without `paths:` loads at launch into every session, like `CLAUDE.md` – reserve that for the rare truly-global rule (e.g. a workspace’s cross-repo contract protocol, Chapter 12). Everything else gets globs, or moves to an orientation map.

Skills

A skill is a directory under `.claude/skills/` containing a `SKILL.md` file. Skills are **repeatable workflows** – task templates that recur frequently enough to deserve a checklist.

When to write a skill

Write a skill when: - The same kind of task arrives 3+ times - The task has multiple steps with verification between them - The cost of skipping a step is high (e.g. forgot to run security review before launching a new auth flow) - A junior team member would benefit from a guided checklist

Don’t write a skill for: - One-off tasks - Tasks where the steps vary too much per instance - Things that are really just a single agent invocation

Skill file structure

```
---
name: <skill-name>
description: <One-sentence description of when to use this skill>
---

# Skill – <Skill Name>

## When to invoke

- Bullet list of trigger conditions

## Workflow

### 1. <Step name>

<2–4 sentences – what to do, what to produce, what to verify>

### 2. <Next step>

...

## Definition of Done

1. <Artifact 1>
2. <Artifact 2>
3. <Verification>

## Cross-links

- `<related agents>`
- `<related rules>`
```

Common skills (most projects benefit from these)

investigate-bug/SKILL.md

Steps: restate bug → identify affected roles → read minimum docs → trace UI → state → API → DB → tests → match touched paths to rules → reproduce → classify severity → propose smallest fix → verify.

build-feature/SKILL.md

Steps: restate feature → identify roles → read minimum docs → run pre-feature checklist (data deltas, RLS, payment impact, etc.) → write acceptance criteria per role → identify rule-covered paths → plan smallest implementation → implement incrementally.

qa-flow/SKILL.md

Steps: identify flows in scope → read testing strategy + relevant orientation map → use browser MCP for UI flows → cover golden path + edge cases + adjacent regressions + cross-role → verify functional + data correctness → severity-rank findings.

compliance-review/SKILL.md

Steps: identify data involved → identify if regulated subjects involved (minors, PHI, EU residents, etc.) → read compliance orientation map + canonical privacy/terms → apply standing checklist (consent, retention, minimization, portability, deletability, processor coverage, disclosure, consent timing, cross-border, marketplace clauses) → flag risks per regulation → produce attorney checklist.

audit-pipeline/SKILL.md

For projects with quality-critical generation pipelines (AI content generation, data ETL, content moderation): trace lifecycle → verify schema → verify required fields on every write → verify validation layers → verify admin review surface → identify gaps → severity-rank findings.

context-refactor/SKILL.md

For maintaining the framework itself: read current INDEX + ledger → categorize suspect content (routing / orientation / rule / skill / agent / outdated / duplicate / conflicting) → move to correct home → consolidate duplicates → flag conflicts (don't silently resolve) → keep root CLAUDE.md small.

Skills vs Rules vs Agents

If the knowledge is...	Put it in...
A path-scoped invariant ("don't do X when editing Y")	Rule
A multi-step workflow that recurs	Skill
A persona with its own scope and Definition of Done	Agent
A one-time decision rationale	PR description
Long-form architecture explanation	Canonical doc
Quick orientation for an area	Orientation map (docs/ai-context/<area>.md)

When something feels like it could be 2 of those: prefer rule > skill > agent (in that order). Rules are most precise (path-globbed). Skills are next (named workflow). Agents are heaviest.

Naming and lifecycle

- Rules: lowercase with hyphens, `.md` suffix. Filename = `name`.
- Skills: lowercase with hyphens for the directory, `SKILL.md` inside.
- New rule from a new gotcha: add to existing rule file if domain matches; create new rule file only if it's a new domain.
- Rule that becomes obsolete (e.g. the underlying constraint was fixed): delete it, don't leave it as "obsolete." Stale rules cause confusion.
- Skill that's invoked < 3x per quarter: consider deleting; the workflow probably wasn't recurrent enough.

06 – Invocation Modes

Six modes coexist by design – the first three are the everyday ones; headless, dynamic workflows and standing routines are for scripts, sweeps and schedules. Pick by task shape, not by habit.

Mode 1: Default `claude` (no flags)

Use for: - Quick questions about code ("what does X do?", "where is Y defined?") - Single-line copy / typo fixes - Pure read tasks (review a file, summarize a function) - One-file bug fixes where the rule + specialist are obvious from the task description - Iterative debugging where you want low-friction back-and-forth

What you get: standard Claude Code main session. Full default tool access (Read, Edit, Write, Bash, all MCP). No `maxTurns` cap. Can spawn any subagent including built-ins (Explore, Plan, general-purpose) and plugin agents.

How to confirm: startup header shows `Claude Code` without an agent name.

Mode 2: Explicit orchestrator `claude --agent <project>-orchestrator`

Use for: - Anything spanning more than one role/domain - Anything spanning more than one feature area - Data integrity, payments, security/privacy, compliance work - Bug investigations where root cause may cross layers (UI → API → DB → infra) - Feature builds where acceptance criteria need writing before implementation - Pre-release QA across multiple roles - Anything where the scope is ambiguous

What you get: orchestrator's full discipline as runtime constraints – `tools` allowlist (no direct Edit/Write; must delegate), `maxTurns` cap, `Agent(...)` restricted to project specialists only. The structured handoff schema is enforced via the orchestrator's instructions.

How to confirm: startup header shows `@<project>-orchestrator`. Confirm visually before starting any production-sensitive task.

Mode 3: Specialist mode `claude --agent <specialist-name>`

Also valid for narrow work in a single domain.

Examples: - `claude --agent frontend-ui` – UI session with browser MCP + Edit/Write - `claude --agent backend-api` – API work with full data tooling - `claude --agent qa-functional` – QA pass with browser MCP + test runners - `claude --agent legal-compliance` – review-only session - `claude --agent database` – schema/migration work

Specialist sessions inherit the specialist's `tools` allowlist and `maxTurns` cap. A specialist whose `tools:` omits `Agent` cannot delegate at all; one that has it should not by framework convention (Chapter 3 anti-patterns – nesting is a platform capability now, see Pitfall 18, not a framework design). Use when you know exactly which domain the work lives in and don't need cross-domain coordination.

Mode 4: Headless `claude -p` (scripts, CI, cross-repo delegation)

`claude -p "<prompt>" --agent <name> --output-format json` runs one turn non-interactively and exits. Verified 2026-08-22: without `--bare`, a `-p` session loads the working directory's `CLAUDE.md`, rules, agents, skills **and runs its hooks** – so it is a faithful way to run a repo's orchestrator from a script or from another repo's session (Chapter 12, Mechanism A). `--json-schema` forces the answer into `structured_output` (use it for the `return:` block); `session_id` in the JSON lets you `--resume` later. `-p` never prompts, so pre-approve tools with `--allowedTools` or a `--permission-mode`, and keep that list narrow. `--bare` is for CI lint-style calls that should load *nothing* from the host; never use it when you want the repo's setup.

Mode 5: Dynamic workflows (many-agent sweeps)

Script-driven orchestration – `agent()`, `parallel()`, `pipeline()` – triggered with the `ultracode` keyword. Use for repeatable fan-outs (review every changed file across N dimensions, migrate 40 call sites, exhaustive research). It complements, not replaces, the orchestrator: the orchestrator decides *what* to do and issues schema-bound handoffs; a workflow is how it runs twenty of them at once. Budget accordingly – a workflow can spawn dozens of agents.

Mode 6: Standing routines (scheduled, unattended)

Mode 4 on a clock, with governance. A routine is a narrow charter file executed on a schedule (a product scheduler, or CI cron + `claude -p`), producing small PRs behind review gates – never direct writes. It is the only mode that runs with nobody watching, which is why it carries the strictest output contract in the framework: repro + truth table on every fix-PR, one reporting channel, budgets and attempt caps, and a human on every merge. Full conventions, starter catalog and budgets: `docs/13-STANDING-ROUTINES.md`.

Why we don't make the orchestrator the default

Setting `agent: "<orchestrator-name>"` in `.claude/settings.json` would force every session into orchestrator mode, which has:

1. **No Edit/Write tools** – every code change requires `Agent(specialist)` delegation. Heavy friction for typos and one-line fixes.
2. `maxTurns` **cap restrictive** for interactive iterative work that often runs 50+ turns.
3. **Body prompt designed as a routing playbook**, not a general-purpose system prompt.
4. **Default Claude Code system prompt is replaced entirely** (per official docs).

The friction cost on everyday work outweighs the marginal enforcement gain. Real always-on enforcement of orchestrator discipline belongs to **hooks** or `permissions.deny` **rules** – both are deferred until you have run 2-3 real tasks through the explicit orchestrator path and confirmed the schema works under load.

Until then: pick the right mode for the task. If you are about to do anything cross-domain or production-sensitive and you are NOT in `--agent <orchestrator-name>`, stop and start a new session with the flag.

Practical recipe

A well-functioning team uses all three everyday modes:

```
# Quick fix (90% of casual work)
cd ~/your-project
claude
# → "Update the button label from 'Sign In' to 'Sign in'"

# Specialist deep-dive (focused domain work)
cd ~/your-project
claude --agent frontend-ui
# → "Audit our Suspense boundaries for client-component leaks"

# Full orchestrated work (multi-domain, production-sensitive)
cd ~/your-project
claude --agent acme-orchestrator
# → "Add a new payment method that requires KYC review and cron-driven activation"
```

Anti-patterns

✗ **Using `claude` for everything.** Cross-domain work flooded with context. Specialists never invoked. Defeats the point.

✗ **Using `claude --agent <orchestrator>` for typos.** Forced delegation overhead. Specialist spawn for a one-line fix.

✗ **Not knowing which mode you're in.** Always check the startup header. If you're not sure, you're probably in the wrong mode.

When you change projects

Each project has its own `.claude/agents/`, so `claude --agent <orchestrator-name>` is project-specific. The orchestrator name should be `<project>-orchestrator` (e.g. `acme-orchestrator`, not just `orchestrator`) so that when you have multiple projects open simultaneously, you can tell at a glance which orchestrator is active.

07 – Folder Structure

How to organize documentation so Claude (and humans) always know where to look.

The three tiers

```
docs/
├── ai-context/                                ← TIER 1: orientation maps (read by agents)
│   ├── INDEX.md
│   ├── HANDOFF_SCHEMA.md
│   ├── ORCHESTRATION_SPOONFEEDER.md
│   ├── PROJECT.md                            ← (v1.2) current truth – what is live where
│   ├── LEARNINGS.md                         ← (v1.2) decisions / failures / corrections
│   ├── GLOSSARY.md                          ← (v1.2) one name per concept
│   └── <area>-experience.md                 ← 50-150 lines per area
├── ARCHITECTURE.md                          ← TIER 2: canonical references (full detail)
├── API.md
├── TECH_STACK.md
├── <UPPERCASE>.md
├── <AREA>_BACKLOG.md                        ← (v1.2) deferred work, one per feature area
├── _archive/                                ← TIER 3: frozen historical
│   ├── README.md                            ← archive convention
│   └── <YYYY-MM>/                          ← dated reports
```

Tier 1 – Orientation maps (docs/ai-context/)

Purpose: Per-area guides Claude reads at the start of any task in that area. Cheaper than reading thousand-line canonical docs.

Format: 50-150 lines per file.

Audience: Agents (primary), humans (secondary).

Naming: <area>-experience.md or <area>-<concern>.md . Examples: - customer-experience.md - admin-experience.md - auth-and-sessions.md - payments.md - realtime-sync.md - data-pipeline.md - infrastructure.md

Body sections: 1. **2-sentence orientation.** What this area is, what it isn't. 2. **Key file paths.** Where to look for the code. 3. **3-5 gotchas worth knowing.** The traps that bit you in production. 4. **Cross-links.** To the canonical docs for full detail and to related rules.

Special files: - INDEX.md – task-type → docs + agent map. The router for the orientation maps. - HANDOFF_SCHEMA.md – the bidirectional handoff schema. - ORCHESTRATION_SPOONFEEDER.md – the human-facing usage guide for the framework. - PROJECT.md , LEARNINGS.md , GLOSSARY.md – (v1.2) the project-truth set (11-PROJECT-TRUTH-AND-LEARNINGS.md). - HOOKS.md (optional) – what the installed hooks enforce (10-HOOK-HARDENING.md). - MIGRATION_LEDGER.md (optional) – tracks doc/structure

migrations in progress. - `LEGACY_BACKUP.md` (optional) – frozen pre-refactor snapshot, if you migrated from a single huge `CLAUDE.md`.

Tier 2 – Canonical references (`docs/<UPPERCASE>.md`)

Purpose: Authoritative source of truth on the topics they cover. Full detail. Updated when the underlying truth changes.

Format: As long as needed. No artificial line cap. A canonical doc may be 500-2000 lines.

Audience: Humans (primary), agents (when deeper detail is needed than the orientation map provides).

Naming: `UPPERCASE.md` . Convention signals “this is canonical.”

Common files: - `ARCHITECTURE.md` – system architecture, components, data flow - `API.md` – endpoint reference (or link to OpenAPI spec) - `TECH_STACK.md` – what’s installed, why, version constraints - `CHANGELOG.md` – release history - `PRODUCT_REQUIREMENTS.md` – PRD - `BUSINESS_DOCUMENT.md` – non-technical product/business context

Cross-referencing: - Orientation maps in `ai-context/` link DOWN to canonical docs. - Canonical docs do NOT link UP to orientation maps (orientation is just a summary; canonical is the truth). - Agents link to either, depending on depth needed.

Tier 3 – Frozen archive (`docs/_archive/`)

Purpose: Historical material that is no longer authoritative but worth preserving for audits.

Format: Whatever the original was – markdown, HTML, PNG, CSV.

Audience: Audits only. Active docs never link here.

Naming: Date-prefixed subdirectories: `<YYYY-MM>/<file>` . Examples: - `_archive/2026-04/SPRINT_2_GOLIVE_2026-04-25.md` - `_archive/2026-04/PRODUCTION_HANDOFF_2026-04-26.md` - `_archive/2025-12/AUDIT_REPORT_2025-12-15.md`

Sub-categories under `_archive/` : - `_archive/2026-04/` – dated reports - `_archive/copilot-audit/` – old AI tool audits - `_archive/screenshots/` – historical UI captures - `_archive/seed-history/` – old seed data - `_archive/html/` – generated HTML reports

Required: `_archive/README.md` documenting: - What this directory is - Why it exists (separation from active docs) - Layout - Three rules: don’t link from active docs, don’t delete, don’t move back to root - “Known link rot” section if you migrated from a structure that had cross-links to now-archived material

What goes where – decision tree

You have a doc to write or move. Where?

Is it a per-area gotcha guide of 50–150 lines?
→ docs/ai-context/<area>.md

Is it the system architecture, API reference, or another full-detail doc?
→ docs/<UPPERCASE>.md

Is it a path-globbed invariant ("don't do X when editing Y")?
→ .claude/rules/<domain>.md

Is it "what is live where" / a decision / a failed approach / a correction?
→ docs/ai-context/PROJECT.md (state) or LEARNINGS.md (history) – see 11-PROJECT-TRUTH-AND-LEARNINGS.md

Is it deferred work ("we'll do that later")?
→ docs/<AREA>_BACKLOG.md – written, never just spoken

Is it a multi-step workflow that recurs?
→ .claude/skills/<workflow>/SKILL.md

Is it an agent persona definition?
→ .claude/agents/<name>.md

Is it a sprint report, audit, post-mortem, or dated snapshot?
→ docs/_archive/<YYYY-MM>/<file>.md

Is it a one-time decision rationale?
→ PR description or commit message – NOT docs/

Is it just routing ("for X tasks, read Y")?
→ CLAUDE.md or docs/ai-context/INDEX.md

Folder-level conventions

Root level

The root contains ONLY: - Tech-stack config files (`package.json` , `tsconfig.json` , `Cargo.toml` , `go.mod` , etc.) - Build/deploy config (`vercel.json` , `Dockerfile` , `docker-compose.yml` , etc.) - CI config (`.github/` , `.gitlab-ci.yml`) - Test runner config (`playwright.config.ts` , `pytest.ini` , etc.) - The framework files (`CLAUDE.md` , `.claude/` , `docs/`) - Application directories (`src/` , `app/` , `lib/` , `pkg/` , `cmd/` , etc.) - Static assets (`public/` , `assets/`)

The root does NOT contain: - Loose orphan scripts (move to `scripts/_archive/` if not actively used) - Screenshots or images (move to `docs/_archive/screenshots/` or `assets/`) - Backup env files (gitignore + delete from history if secrets were committed) - Generated artifacts (gitignore) - Stub directories with one file (consolidate elsewhere)

Tracked-cache cleanup

These directories are commonly tracked by accident and should be gitignored: - Tool MCP caches (`.playwright-mcp/` , `.copilot-audit/`) - Test runner per-run state (`test-results/` , `coverage/`) - Database CLI temp dirs (`supabase/.temp/` , `firebase/.firebase/`) - Build artifacts (`.next/` , `dist/` , `build/` , `__pycache__/`) - Local logs (`logs/`)

If they're already tracked: `git rm -r <dir>` and add to `.gitignore` in the same commit.

Env file safety

Add broad gitignore patterns for env file variants:

```
.env
.env.*
!.env.example
.env*.bak*
.env*staging_tmp
.env*tmp
```

If env files with secrets are already in git history, that's a separate workstream – rotate secrets first, then scrub history with `git-filter-repo` or BFG. Removing the working-tree copy doesn't remove the history.

Variants for non-standard projects

Monorepo

```
monorepo-root/
├── CLAUDE.md                                ← root router; mentions per-package CLAUDE.md exists
├── .claude/
│   ├── agents/                             ← orchestrator + cross-package specialists
│   ├── rules/                             ← cross-package rules
│   └── skills/
├── docs/                                    ← cross-package docs
│   └── ai-context/
├── packages/
│   ├── package-a/
│   │   ├── CLAUDE.md                      ← package-specific router
│   │   └── .claude/agents/                ← package-specific specialists (optional – visible
only to sessions started in this package; see 02 § Monorepos)
│   │   └── docs/ai-context/              ← package-specific orientation
│   └── package-b/
│       └── ...
```

Multi-product in one repo

```
your-product-repo/
├── .claude/agents/
│   ├── <repo>-orchestrator.md
│   ├── product-a-frontend.md
│   ├── product-a-backend.md
│   ├── product-b-frontend.md
│   └── shared-database.md
├── docs/ai-context/
│   ├── INDEX.md
│   ├── product-a-experience.md
│   └── product-b-experience.md
```

← lists tasks per product

Mobile + web shared

```
your-app/
├── apps/
│   ├── web/
│   └── mobile/
├── packages/
└── .claude/agents/
    ├── web-ui.md
    ├── mobile-ui.md
    ├── shared-lib.md
    └── backend-api.md
```

← Next.js / Vite / etc.
← React Native / Flutter / native
← shared libs

← only edits apps/web/
← only edits apps/mobile/
← only edits packages/
← stays separate if hosted elsewhere

08 – Common Pitfalls

Twenty-eight hard-won lessons from real-world adoption. Read this before bootstrapping a project.

Pitfalls 1-17 date from v1.0/v1.1. Pitfalls 18-26 were added in v1.2.0 after three further months of production use; Pitfall 18 also corrects two v1.0 claims that the platform has since made false. Pitfalls 27-28 were added in v1.3.0 alongside Chapter 13 (standing routines).

Pitfall 1: Making the orchestrator the project default

Setting `agent: "<orchestrator-name>"` in `.claude/settings.json` SEEMS like the right move ("now everyone gets the discipline always-on!"). It is not.

The orchestrator has: - No Edit/Write tools (delegations required for every change) - A modest `maxTurns` cap (insufficient for interactive iterative work) - A body prompt designed for routing, not general work - A system prompt that REPLACES the default Claude Code system prompt entirely

Daily work (typo fixes, "what does this function do?", quick prototyping) becomes painful.

Right answer: Don't set it as default. Document the invocation modes (Chapter 6) in your spoonfeeder. Train the team to use `--agent <orchestrator-name>` for production-sensitive cross-domain work and plain `claude` for everything else.

Pitfall 2: Enabling memory on REVIEW-ONLY agents

Per the official Claude Code subagents docs, when you set `memory: project` (or `user` / `local`), the harness automatically grants Read/Write/Edit so the agent can manage its memory files.

This **silently bypasses your** `tools: allowlist` – a `legal-compliance` agent you carefully restricted to Read/Grep/Glob suddenly has Write/Edit because you turned on memory.

Right answer: Never enable `memory` on REVIEW-ONLY agents. For implementation specialists where memory is useful, enable it explicitly and document that the auto-Write/Edit is intentional.

Pitfall 3: Treating documentation enforcement as runtime enforcement

The handoff schema, the universal evidence rule, the failure_condition observation, the soft hop limits – these are **documentation discipline**. They work because agents follow their instructions. They do NOT physically prevent a misbehaving agent from emitting a malformed handoff.

What IS runtime-enforced (by the Claude Code harness): - `tools: allowlists` - `disallowedTools: denylists` - `maxTurns: caps` - `Agent(name1, ...)` allowlists when orchestrator runs as main thread - `mcpServers:` per-agent MCP scoping - Subagent context isolation

What is NOT runtime-enforced: - The handoff YAML block being present - The evidence rule being followed - Refusal of vague delegations - Hop limits ("3rd same-specialist delegation → escalate") - "Rules read before edit" - Definition of Done completeness

Right answer: Be honest about which layer enforces what. If you need hard enforcement of a documentation rule, write a PreToolUse hook on the `Agent` tool that validates the YAML block before letting the call proceed. Don't pretend documentation is enforcement.

Pitfall 4: `Agent(name1, name2, ...)` only binds in main-thread mode

The `Agent(specialist-1, specialist-2, ...)` allowlist on the orchestrator's `tools` field restricts which agents the orchestrator can spawn. **This restriction only takes effect when the orchestrator runs as the main thread** via `claude --agent <orchestrator-name>`.

When the orchestrator is spawned AS a subagent from another session (via the `Agent` tool), the `Agent(...)` allowlist on its `tools` field is **not** what governs it – the spawning session's permissions are. (v1.0 of this chapter said subagents cannot spawn subagents at all; that is no longer true – see Pitfall 18. Nesting is now allowed to a configurable depth, which makes the allowlist question *more* important, not less.)

When the main session reads CLAUDE.md and acts AS-IF the orchestrator (without `--agent`), the allowlist has no enforcement at all.

Right answer: Document the limitation in the orchestrator file. For hard enforcement on the main thread without `--agent`, use `permissions.deny` in `.claude/settings.json`:

```
{
  "permissions": {
    "deny": [
      "Agent(Explore)",
      "Agent(Plan)",
      "Agent(general-purpose)",
      "Agent(<plugin>:*)"
    ]
  }
}
```

This blocks the main session from spawning built-ins/plugins regardless of which agent it's posing as.

Pitfall 5: MCP allowlist brittleness – list servers, not individual tools

If your specialist needs Chrome DevTools MCP and Playwright MCP, you might be tempted to list every individual tool:

```
tools: Read, Edit, Bash, mcp__chrome-devtools__navigate_page, mcp__chrome-devtools__click,
      mcp__chrome-devtools__take_screenshot, mcp__chrome-devtools__list_console_messages,
      ... (28 more)
```

This is brittle. When the MCP server adds a new tool, your specialist won't get it. When tool names change, your specialist breaks silently.

Right answer: Use the documented wildcard syntax (per Claude Code permissions docs):

```
tools: Read, Edit, Bash, mcp__chrome-devtools__*, mcp__playwright__*
```

The wildcard covers all current AND future tools from that MCP server. Other MCP servers (e.g. Supabase MCP, your custom MCP) remain blocked because they're not in the allowlist.

Pitfall 6: Dropping `tools:` for "convenience"

It's tempting to omit `tools:` on a specialist so it inherits everything from the parent. Don't.

Missing `tools:` means the specialist has full access to: - Every MCP server you've installed (database access, deploy access, third-party APIs) - Every built-in tool

For specialists that legitimately need broad MCP access (e.g. `qa-functional` needs browser MCP), use the wildcard pattern from Pitfall 5. For specialists that don't need MCP, list only the standard tools.

Right answer: Always declare `tools:` explicitly. Defense-in-depth via least privilege.

Pitfall 7: Ignoring `.env*` history

If env files with secrets are already in git history (you committed `.env.local.bak` once, or `.env.staging_tmp` slipped through), removing the file from the working tree doesn't remove the secrets from history.

`git log -p -- .env*` will show every secret that ever leaked. So will any clone of the repo. So will GitHub's commit history.

Right answer: 1. Rotate every credential that was in any leaked env file (treat them as compromised). 2. Use `git-filter-repo` or BFG Repo-Cleaner to scrub the files from history. 3. Force-push the scrubbed history (this is destructive – coordinate with the team). 4. Add broad gitignore patterns to prevent recurrence: `.env*.bak*`, `.env*staging_tmp`, `.env*tmp`.

Pitfall 8: Reorganizing docs without a link sweep

Moving docs is mechanically easy (`git mv`). Verifying links isn't. If you move `docs/AUDIT_2025-12.md` → `docs/_archive/2025-12/AUDIT_2025-12.md` and don't check, you'll discover the broken refs weeks later when someone follows a link in CHANGELOG and gets a 404.

Right answer: Before any doc reorg, capture a baseline of all `docs/*.md` and `.claude/*.md` references:

```
grep -rohE "(docs|\.claude)/[A-Za-z0-9_-]+\.\md" CLAUDE.md docs/ .claude/ | sort -u > /tmp/refs-before.txt
```


After the move:

```
for path in $(cat /tmp/refs-before.txt); do
  [ -f "$path" ] || echo "BROKEN: $path"
done
```

Fix every broken ref before committing the move. Or do the reorg in phases with the sweep between each phase (this is what we did).

Pitfall 9: Treating Claude's training data as authoritative

Claude's training data has a cutoff. Library APIs, SDK signatures, and platform conventions change. Even for things Claude "knows" – verify against current docs before writing code.

Right answer: When writing or refactoring code that touches a library/SDK/platform, instruct the agent to verify against current docs first. Pin doc URLs in the orientation map for that area. Add a rule: "Before writing X-SDK code, fetch the current docs at ."

Pitfall 10: Letting the orchestrator do all the work

If the orchestrator finds itself doing the actual implementation, you've drifted from the pattern. The orchestrator should: - Read enough to delegate well - Issue handoffs - Aggregate returns - Decide if more delegation is needed

The orchestrator should NOT: - Edit code (it has no Edit/Write tool by design) - Run long verifications itself (delegate to qa-functional) - Make architecture decisions without consulting product-flow

Right answer: If the orchestrator is doing implementation, your specialists are too narrow. Either broaden a specialist's scope or add a missing one.

Pitfall 11: Direct push to the integration branch bypassing PR review

Direct pushes feel faster. They skip: - CI / staging deploy verification - Vercel / Cloudflare / Netlify preview URLs - Code review by humans - Auditable PR history

For trivial doc changes the cost of a PR is low. For multi-file reorgs, the cost of a broken merge to the integration branch (`develop` , `main` , whatever your base branch is) is high (other engineers' worktrees, broken builds, lost time).

Right answer: Use direct push to the integration branch ONLY when: - The change is < 5 files AND - The change is doc-only OR adds purely-additive code AND - You have explicit authorization in the current session

For everything else: push the branch, open a PR, let the preview deploy run, merge.

Pitfall 12: Not verifying after the work is “done”

A specialist's `status: completed` is the agent's claim, not a guarantee. The orchestrator's job before merging is to verify: - `verified_claims` covers everything in the original `claims` list - `unverified_claims` is empty OR explicitly accepted as future-work - `do_not_pass_downstream_without_verification` items are flagged for the next hop - `tests_run` actually includes the relevant tests - `files_changed` matches `expected_artifacts`

Right answer: Treat the return schema as a contract. If a specialist returns `status: completed` but `verified_claims` is empty, push back: “What did you actually verify?” Specialists should not be allowed to claim done without showing work.

Pitfall 13: Skipping the `failure_condition` field

`failure_condition` feels redundant with `verify_before_acting` – until a specialist mid-task discovers the orchestrator's whole premise was wrong and doesn't know whether to push through or stop.

Without `failure_condition`, specialists tend to push through (sunk-cost on partial work). With `failure_condition`, they have an explicit STOP signal.

Right answer: Make `failure_condition` required in your handoff schema. Reject delegations that omit it. The discipline of articulating “what would prove me wrong” is itself valuable – if the orchestrator can't name a failure condition, the task is too vague.

Pitfall 14: Letting `docs/` rot without a librarian

Without active maintenance, `docs/` accumulates: - Sprint reports from 6 months ago - Audit reports referenced by no one - Outdated architecture decisions - HTML/PNG artifacts - Dated migration plans

After a year, `docs/` has 50+ files and engineers can't tell which are authoritative.

Right answer: Have a `context-librarian` specialist whose job is exactly this. Schedule a quarterly cleanup pass (or more often). Use the `context-refactor` skill to guide the work. Move stale material to `docs/_archive/<YYYY-MM>/`. Never delete (you might need it for an audit).

Pitfall 15: One huge CLAUDE.md instead of a router

If your CLAUDE.md grows past 200 lines, you're using it wrong. CLAUDE.md is the router; detail goes elsewhere.

A 2000-line CLAUDE.md means: - Every Claude Code session loads 2000 lines into context up front - The router and the detail are mixed (you can't tell what's truth vs. what's a passing note) - Updates require touching the same file from many directions - You lose the ability to scope rules to file paths (everything applies everywhere)

Right answer: Refactor a long CLAUDE.md into the framework's tiers. Move per-area gotchas to `.claude/rules/<domain>.md` (with `paths: globs`). Move orientation to `docs/ai-context/<area>.md`.

Move full detail to `docs/<UPPERCASE>.md`. Keep `CLAUDE.md` as the routing index – golden rules + workflow + tables + cross-links.

We did this on a real codebase. The pre-refactor `CLAUDE.md` was 2,928 lines. The post-refactor one is ~150. Every existing rule survived; nothing was lost. The routing pattern made it dramatically easier for both Claude and humans to navigate.

Pitfall 16: Bootstrap on a repo with existing Claude Code config

If you ran `/init` previously, or your team has been adding to `CLAUDE.md` for months, `BOOTSTRAP-PROMPT.md` must NOT silently overwrite that work.

Right answer: the prompt now includes mandatory pre-flight checks at the top – snapshot to `.claude-pre-bootstrap-backup/`, naming-collision detection, `paths:` glob conflict detection, drift detection on existing `CLAUDE.md`, and a decision gate that STOPS if any pre-flight raised a `<NEEDS USER CONFIRMATION>` flag.

Specific risks: - Existing `CLAUDE.md` with team rules → Pre-flight 4 (drift detection) flags stale content; Step 9 (merge step) shows a 3-pane diff before writing. - Existing `.claude/agents/<name>.md` with same name as a proposed specialist → Pre-flight 2 (naming collision check) STOPS for explicit user decision per file. - Existing `.claude/rules/` with overlapping `paths:` → Pre-flight 3 detects glob overlap; both rule files would be referenced creating contradictory guidance. - Existing `.claude/agents/` with personas that overlap proposed specialists → Pre-flight 5 surfaces the parallel system; user decides migrate vs coexist.

The pre-flight workflow is non-optional – even on apparent greenfield projects, run it. Cost is 30 seconds; benefit is never silently destroying team work.

Pitfall 17: Hooks installed mid-session don't activate until the next session

When you add hook entries to `.claude/settings.json` (per `docs/10-HOOK-HARDENING.md`), the running Claude Code session does **not** pick them up. `settings.json` is loaded once at session start.

This is the single most surprising property of hook deployment. We watched a session install a `PreToolUse` rule-surfacing hook, manually verify the script worked end-to-end with synthetic stdin, then edit a file matching a rule glob – and the hook didn't fire. The hooks were correctly written and wired; they just weren't loaded.

Right answer: 1. After installing hooks, **end the current session** (`/exit` or close the tab). 2. Start a fresh Claude Code session. 3. Verify the hook fires by intentionally triggering it (e.g. edit a file matching a rule glob and look for the `<system-reminder>` block in context). 4. Only declare the hook deployed after a fresh-session smoke test passes.

If you're shipping hooks in a PR, write the verification recipe into the PR description so reviewers don't have to re-derive it. The `templates/hooks/HOOKS.md.template` includes a "Verifying hooks are live"

section that's safe to copy to your project's `docs/ai-context/HOOKS.md` .

A side effect of this: never auto-merge a PR that adds hooks. The author should restart their session and verify before approving.

Pitfall 18: The platform moves under your conventions – re-verify the docs every quarter

Two claims this framework made in v1.0 became false within three months, and a third convention became redundant:

- **"Subagents cannot spawn subagents."** They can – by default up to three layers below the main conversation (`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` changes it; at the limit the `Agent` tool is withheld). The framework's *design* choice – specialists return `recommended_next_agent` rather than chaining – still stands, because a flat tree is auditable and a deep one is not. But it is a choice now, not a platform limit, and an orchestrator that nests must carry the handoff schema across every hop.
- **"Rules need a custom `applies_to:` convention plus a hook."** `.claude/rules/*.md` with a `paths:` frontmatter field is native. Rules without `paths:` load at launch; rules with it load when Claude *reads* a matching file; nested `.claude/rules/` in subdirectories load on demand; symlinked rule files are supported. The framework's rule template now uses `paths:` . The rule-surfacing hook (Pattern 1) survives for one gap only – a brand-new file that is *written* without ever being *read* matches no `paths:` rule – and reads the same field.
- **Dynamic workflows.** Script-driven fan-out of many subagents (`agent()` / `parallel()` / `pipeline()`) is a documented feature, triggered by the `ultracode` keyword. It does not replace the orchestrator (it is for repeatable, many-agent sweeps), but an orchestrator that needs twenty parallel readers should use it rather than hand-spawning.

Right answer: every framework claim about the platform carries a *verified-on* date. The REFINEMENT prompt now includes a "platform drift" pass: re-read the subagents, memory, hooks and headless pages and diff them against Chapters 3, 5, 6, 10 and 12. Treat a claim older than a quarter as `documented-unverified` (Chapter 11) until re-checked.

Pitfall 19: A rule file is a claim, not evidence

A rule file documented the wrong HTTP verb for a route. Three agents trusted it, shipped a client that called the wrong verb, and every user of that flow saw a generic "check your connection" message for a week. The fourth agent read the route's `export` line.

Rules are written by people and agents at a point in time. They rot like any other document – faster, because they are short and confident.

Right answer: a rule is `documented-unverified` until you have looked at the thing it describes. Before *relying* on a documented fact to build something, check it at the source (the route, the schema, the config). When a rule is found wrong, fix it in the same turn – never leave a known-false rule standing because "it's not my task".

Pitfall 20: Deferred work that lives only in prose vanishes

“We’ll do the retry logic in a follow-up” was said at the end of a session. No follow-up happened. Two weeks later the missing retry was reported as a bug, investigated from scratch, and fixed – with none of the context the first session had.

Right answer: a turn that names deferred work may not end until that work is appended to `docs/<AREA>_BACKLOG.md` with what / why / effort / revisit-when (Chapter 11). The backlog id is checked against the remote and open PRs first – two concurrent sessions took the same id on the same day. Verbal follow-ups are not follow-ups.

Pitfall 21: A production push that does not freshen the docs strands the next agent

A feature went to production on Tuesday. Its orientation map still described it as staging-only behind a flag. On Thursday a fresh agent “enabled” it – and re-opened a migration that had already been applied.

Right answer: every production push updates the full doc set *in the same turn*: `docs/CHANGELOG.md` (the anchor), the affected orientation maps, the affected rules, and `PROJECT.md` §3 if the production state changed. Documentation discipline missed this about one push in five; it is now a Stop hook (`doc-freshness-gate` , Chapter 10 Pattern 5). A future agent sees only what is in git.

Pitfall 22: Correction-capture regexes false-fire on benign phrases – and the cost is trust

The correction-capture hook (Pattern 2) fired on “*You already have access to it*” – a perfectly polite sentence – because its regex matched `you already` . It also fired on a test plan that *quoted* a correction phrase, and on a framework doc that *explained* the hook.

Right answer: three guards, all now in the template. (1) Anchor the frustration verbs to what follows them (`you already (did|changed|broke)`), never bare. (2) Strip code fences, inline code and heredoc bodies before matching – quoted text is data, not a correction. (3) Keep the loop guard (`stop_hook_active`) so a false fire costs one reply (“not a correction”), never a trapped session. When a false fire does happen, tighten the pattern the same day; a hook that cries wolf gets disabled by the team within a week.

Pitfall 23: A killed check is inconclusive, not failed

The build-gate Stop hook capped the build at five minutes. On a machine also running two other builds, a legitimate four-minute build was killed every time – and reported as *failed*, with a warnings-only tail the agent then tried to “fix”. An alarm loop with nothing to fix.

Right answer: distinguish `status === null` (killed by timeout or signal) from a non-zero exit. A killed run has produced **no failure evidence**; exit 0 silently and let CI (which builds every PR anyway) be the

authority. Only a real non-zero exit blocks the stop. Size the cap to the slowest honest build on a busy machine, not to the fastest one on an idle machine.

Pitfall 24: Many sessions, one working directory

Three agent sessions shared one checkout. One was launched into a stale, detached snapshot of the tree and “fixed” code that had already been rewritten. Two ran the build at once and corrupted each other’s output directory – every route 500’d with an error that read like a code bug and was pure directory collision. Two picked the same migration number.

Right answer: `git fetch` and compare to the remote base *before the first edit* – the tree you were launched in is not trustworthy. Do feature work in an isolated worktree branched from the fresh-fetched base (`claude --worktree` creates one; symlink the dependency directory rather than duplicating it). Never run a build and a dev server in the same directory at once. Before taking any sequential id (migration number, backlog id, schema version), check the remote *and* other sessions’ open PRs.

Pitfall 25: Never report a negative from a reader you have not verified can see the whole set

“The generator refused these 336 records” turned out to mean “the reader that listed the records was silently capped at 1,000 rows and never showed the generator 25% of them.” A confident negative result, wrong, because the *reader* had a ceiling nobody checked.

Most data-access layers cap a bare query (ORMs, REST layers, search APIs – 1,000 is a common default), and an explicit `limit` above the cap often does not raise it. No error, no truncation flag.

Right answer: before reporting “none found”, “it refused”, or “zero exist”, establish the reader’s ceiling. Page with a stable order, aggregate in the database, or use a count query. A capped read is worse than a failed one: it looks like a confident answer.

Pitfall 26: Two words for one thing ships bugs

The same concept was called `domain` in the database, `topic` on one component and `chapter` on the page – and `topic` *already meant something else* in a legacy subsystem. A gate took the new meaning, looked it up where the old meaning lived, matched nothing, and an empty match silently widened a filter to the whole dataset. Every user of that gate got off-target results for a day.

Vocabulary drift is not cosmetic. It is how a correct-looking lookup returns the wrong set.

Right answer: one glossary (`docs/ai-context/GLOSSARY.md` – a rule file, not a nicety) naming each concept exactly once, with the DB column, the type field and the UI label that carry it. Adding a fifth name for an existing concept is a defect. Rename only while already editing the file that carries the wrong name, and say so in the commit.

Pitfall 27: A context system is a program – profile it and bisect it

The rules, router and hooks you build with this framework are injected into sessions as tokens, and token cost compounds invisibly: a mature install can front-load thousands of lines into every session and nobody notices until the bill or the drift does. Worse, misbehavior gets misattributed – when the agent rabbit-holes or fixates, teams blame the model while a stale rule or an over-broad skill is doing the steering.

Right answer: treat context like code, with two operational habits. **Bisect:** reproduce the misbehavior with project context disabled (`claude --safe-mode`); if it disappears, the fault is in your CLAUDE.md/rules/skills – find it by re-enabling halves. **Weigh:** once a quarter, measure what the install injects per session (`/usage` breaks down where tokens go; runaway loops, extreme parallelism and one inefficient skill are the usual culprits) and make every rule earn its tokens – REFINEMENT check 10 makes this a standing pass. A rule that has never changed an outcome is not free; it is paid for on every session.

Pitfall 28: An unattended job without a verified retire path runs forever

A scheduled pipeline ran for 17 days without a single job completing. The completion write violated a database CHECK constraint, its error return was never read, the job stayed “running”, and a reaper re-queued it – so the system burned its full capacity re-doing satisfied work, while every dashboard showed green because runs *were happening*. Nothing a person watched distinguished “ran” from “worked”.

Right answer: for any unattended loop (a cron, a queue drainer, a standing routine – Chapter 13): the completion write’s error is READ, and a failed completion is a loud failure, not a silent retry; attempt caps park a grinding job instead of letting it spin; and the reporting surface states what was *verified done*, never just that the process exited. “The routine ran” is a claim about the scheduler; only the checked completion write is a claim about the work.

09 – Runbook: Bootstrap a New Project

Step-by-step guide for adopting this framework in a real codebase. Plan for ~2-4 hours of focused work.

Prerequisites

- Claude Code installed (`claude --version` should print v2.x or later)
- A project repository (any tech stack)
- This framework cloned somewhere on your disk (the quickstart uses `~/frameworks/claude` ; any path works – you paste it into the prompts as `<framework path>`)
- 2-4 hours of focused time (don't rush this; the foundation lasts years)

Phase 0 – Decide if you need this framework (10 min)

Skip this framework if: - Your project is a solo prototype - Your project has fewer than 4 distinct domain areas (frontend, backend, etc.) - Your team uses Claude Code for the occasional question, not as a development partner - Your project is < 5,000 lines of code

Use this framework if: - Your project has multiple roles (admin, customer, partner, etc.) - Multiple data systems are in play (DB + cache + search + queue) - Real compliance/security/payments concerns exist - A team uses Claude Code daily on the same codebase - You've experienced cascading hallucinations or context flooding

Phase 1 – Inventory your project (30 min)

Open Claude Code in your project root, on a fresh branch (the setup only adds files, but a clean branch makes it easy to throw away):

```
cd ~/your-project
git checkout -b setup/claude-orchestration
claude
```

Paste `prompts/INVENTORY-PROMPT.md` (from this framework), replacing `<framework path>` . Claude will: - Scan your codebase structure - Propose a list of specialists based on your domain boundaries - Surface clutter that should be archived - Identify your protected branches (`main` , your integration branch, etc.) - Identify your tech stack and which MCP servers will be useful

You answer: confirm/adjust the proposed specialist list. This is the foundation of everything that follows.

Phase 2 – Bootstrap the framework files (45 min)

In the same Claude Code session, paste `prompts/BOOTSTRAP-PROMPT.md` . Reference this framework's path so Claude can read templates:

The framework lives at <framework path> (e.g. ~/frameworks/claude).
Read templates from there. Customize for this project: <project-name>.

Claude first runs the pre-flight safety checks (backup of any existing `CLAUDE.md` / `.claude/` to `.claude-pre-bootstrap-backup/`, collision and drift detection – Pitfall 16), then will: 1. Create `.claude/agents/<project>-orchestrator.md` from `templates/orchestrator-agent.md.template` 2. Create one `.claude/agents/<specialist>.md` per specialist, from the appropriate template 3. Create `docs/ai-context/HANDOFF_SCHEMA.md` from the template 4. Create `docs/ai-context/INDEX.md` with task → docs map 5. Create `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md` with usage guide 6. Create skeleton `docs/ai-context/<area>-experience.md` files (one per major domain) 7. Create `CLAUDE.md` at root (the router, shaped by `templates/CLAUDE.md.template`; merged with a 3-pane diff if one already exists) 8. Create `docs/_archive/README.md` 9. Update `.gitignore` with framework-relevant entries 10. (v1.2) Create the project-truth set – `docs/ai-context/PROJECT.md`, `LEARNINGS.md`, `GLOSSARY.md`, the `.claude/skills/<project>-engineering/SKILL.md` playbook, and one `docs/<AREA>-BACKLOG.md` 11. Create the `.claude/rules/<domain>.md` files and the two starter skills the inventory proposed (Phases 3-4 below are where you review and extend them)

You verify each file before saving. Claude shouldn't ship anything you haven't read.

Phase 3 – Add your first rules (30 min)

The hardest part of this framework is writing useful rules. They take real domain knowledge. Bootstrap created the rule files the inventory proposed; this phase is where you read them critically and add what the inventory missed.

In the Claude Code session, ask:

Based on the codebase scan, propose 3-5 rule files for `.claude/rules/`.
For each, list the path globs (paths:) and 2-3 hard rules with WHY + HOW TO APPLY.
Do not create any rule that's just style preference – only invariants where breaking them caused or could cause a real production issue.
Wait for my approval before creating any files.

Review each proposal. Approve, edit, or reject. Common rule files to start with: - `backend-api.md` (or your equivalent server-side concern) - `frontend-ui.md` (or `mobile-ui.md`) - `database.md` - `auth-security.md` - `testing.md`

Aim for 3-5 rules per file at first. You'll add more over time as production teaches you new gotchas.

Phase 4 – Add 1-2 starter skills (20 min)

Most projects benefit from at least these skills (bootstrap creates them alongside the `<project>-engineering` playbook; review them here, or ask for them if they were skipped): - `investigate-bug/SKILL.md` - `build-feature/SKILL.md`

Ask Claude:

Create `.claude/skills/investigate-bug/SKILL.md` and `.claude/skills/build-feature/SKILL.md` based on `templates/skill.md.template`, customized for this project's tech stack and roles.

You can add more skills later as patterns recur (qa-flow, compliance-review, audit-pipeline, context-refactor).

Phase 5 – Verify (15 min)

Run these checks:

```
# All agents register (or type /agents inside a session)
claude agents

# Doc-link sweep – find broken refs
grep -rohE "(docs|\.claude)/[A-Za-z0-9_-]+\.\md" CLAUDE.md docs/ .claude/ | sort -u | while
  read p; do [ -f "$p" ] || echo "BROKEN: $p"; done

# Build still passes (whatever your build command is)
npm run build # or: pytest, go build, cargo build, etc.
```

Fix any breakage before moving on. Common issues: - Agent file YAML invalid (claude agents will tell you) - Markdown link in CLAUDE.md or INDEX.md to a file that doesn't exist (typo) - Rule glob points to a path that doesn't match anything (mostly harmless but confusing)

Phase 6 – Run a real task through the orchestrator (30 min)

Pick a real task – ideally a small bug or a small feature. Open a NEW Claude Code session in orchestrator mode:

```
cd ~/your-project
claude --agent <project>-orchestrator
```

Verify the startup header shows `@<project>-orchestrator`.

Give it a real task. Watch what happens: - Does the orchestrator emit the outbound handoff YAML block? - Do specialists return the inbound YAML block? - Does the orchestrator catch any rejected_claims correctly? - Does verification (tests, build) actually run?

If anything is off: - Tighten the orchestrator's "outbound discipline" section - Tighten the specialist's "incoming handoff validation" section - Add a missing rule - Add a missing orientation map

Don't add hooks yet. Get the documentation enforcement working through 2-3 real tasks first.

Phase 7 – Commit, open a PR, document for your team (20 min)

```
git add CLAUDE.md .claude/ docs/ .gitignore      # the backup folder is gitignored
git commit -m "chore: bootstrap Claude Code orchestration framework"
git push -u origin setup/claude-orchestration
gh pr create --base <your-base-branch> --title "Bootstrap Claude orchestration framework"
```

Add to your project's existing onboarding docs: - "We use Claude Code with a custom orchestration setup. See `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md` ." - "Always pick the right invocation mode: `claude` for quick work, `claude --agent <orchestrator>` for cross-domain work." - "Don't push to main without project-owner approval."

Optional: announce in your team chat with a 30-second demo of the orchestrator handling a multi-step task.

Phase 8 – Add hardening as needed (optional, after 2 weeks)

After running real tasks for a couple weeks, decide if you need any of the five shipped hook patterns (`docs/10-HOOK-HARDENING.md` + `templates/hooks/` – rule-surfacing, correction-capture, build-gate, lint-fix, doc-freshness gate; install into `.claude/settings.json` , then restart the session – Pitfall 17), or one of these bespoke ones:

Hop limits via PreToolUse hook

If you see the orchestrator delegating to the same specialist 3+ times in a row without progress, add a hook that counts Agent invocations per turn and warns past N hops.

Hard schema enforcement via PreToolUse hook

If specialists keep emitting malformed return blocks, add a hook that validates the YAML before letting the call complete.

`permissions.deny` for built-ins/plugins on main thread

If team members keep accidentally invoking `Explore` / `Plan` /plugin agents from default `claude` sessions when they should be in orchestrator mode, add to `.claude/settings.json` :

```
{
  "permissions": {
    "deny": [
      "Agent(Explore)",
      "Agent(Plan)",
      "Agent(general-purpose)",
      "Agent(<plugin>:*)"
    ]
  }
}
```

Per-agent memory for cross-session learning

For implementation specialists where institutional knowledge accumulates (debugging recipes, schema gotchas), add `memory: project` to the agent frontmatter. **Do NOT enable for REVIEW-ONLY agents** – see Pitfall 2.

Phase 9 – Quarterly maintenance

Every 3 months, dedicate a session to: 1. Run the `context-refactor` skill (or just have the `context-librarian` specialist do a sweep) 2. Move stale dated reports to `docs/_archive/<YYYY-MM>/` 3. Reconcile any rule conflicts 4. Update orientation maps for areas where reality has drifted 5. Verify all agents still register after dependency updates 6. Run `prompts/REFINEMENT-PROMPT.md` – it re-checks platform drift (Pitfall 18) and `PROJECT.md` freshness

Common time-sinks to avoid

- **Don't try to make every rule perfect on day 1.** Ship 5 rules; add 1 per month as production teaches you.
- **Don't over-specialize.** 6-8 specialists is plenty. Splitting too narrowly creates routing confusion.
- **Don't write canonical docs from scratch in this framework.** Use what you already have. The framework adds the orientation/router layer on top.
- **Don't manually merge agent files when adding a rule.** Always edit the rule file; agents reference it via `paths`.

When to stop and ask

- If `claude agents` shows fewer agents than you expect → YAML frontmatter syntax error
- If specialist outputs don't include the return YAML block → the specialist's body needs the "Return schema" section
- If the orchestrator keeps delegating in a loop → check `failure_condition` is articulated
- If a specialist refuses a delegation → check the orchestrator's outbound block has all required fields

End state

You have: - A `<project>-orchestrator` and 4-10 specialists - 3-5 rule files covering your highest-risk paths - 2-4 skills for recurring workflows - Three-tier docs (orientation maps + canonical refs + archive) - A team that knows how to pick invocation modes - Auditable handoffs for every cross-domain task

Time invested: 2-4 hours setup + ~30 min/quarter maintenance.

Time saved: every cascading-hallucination incident you DIDN'T have. Every cross-domain task that got the right specialists without you thinking about it. Every new engineer who could navigate the codebase without 2 weeks of pairing.

10 – Hook-Based Hardening (Optional)

The framework’s documentation discipline (handoff schema, evidence rule, path-globbed rules, refusal contracts) covers ~95% of the value at 5% of the complexity. **Most projects should never add hooks.**

But Pitfall 3 was honest: *“If you need hard enforcement of a documentation rule, write a `PreToolUse` hook on the `Agent` tool that validates the YAML block before letting the call proceed. Don’t pretend documentation is enforcement.”* This chapter is the runbook for *when* you need that hard enforcement, *which* hooks are worth adding, and *how* to keep them small.

This chapter is **optional**. Skip it on adoption. Come back when you have evidence that documentation discipline is being skipped under real-world pressure.

When to add hooks

Add a hook only when you have **concrete evidence** that a documentation rule is being skipped – not pre-emptively, and not because hooks “feel rigorous.”

Concrete evidence looks like:

Signal	Indicates
The same correction repeats across multiple sessions (“you used the wrong helper again”)	Rule exists but isn’t being read
Auto-memory or PR review surfaces the same pattern repeatedly	Documentation discipline is failing
Bugs ship to production despite a rule that explicitly forbade them	Honor-system enforcement isn’t working
New team members regularly miss the same gotchas after onboarding	The rule exists but the trigger to read it doesn’t
<code>npm run build</code> / <code>pytest</code> / equivalents are skipped on commits ~10%+ of the time	Pre-commit human discipline is unreliable

If none of these apply, stay on documentation discipline. The hook layer adds: - Per-session-start startup cost (settings.json must be loaded) - A small surface area of fail-paths (if a hook script throws, you need to fail-silent or you block all edits) - Maintenance: scripts must be kept generic enough to survive code reorganization - Cognitive load: contributors have to know hooks exist and what they do

That cost is justified **ONLY** when documentation enforcement has demonstrably failed.

The five hook patterns worth knowing

These are tested patterns, ordered by leverage-per-line-of-code.

v1.2.0 note on Pattern 1. `.claude/rules/*.md` with `paths:` frontmatter is now loaded by the platform itself when Claude *reads* a matching file (Pitfall 18). That covers most of what this hook did. Keep the hook for the one documented gap – a new file *written* without a prior read – and point it at the same `paths:` field so there is one source of truth. If your team never writes files blind, delete the hook.

Pattern 1 – `PreToolUse` rule-surfacing (highest leverage)

Closes: the gap where agents skip “before editing, read the matching `.claude/rules/*.md`.” This rule appears in every framework deployment and is the most-skipped one under deadline pressure.

Mechanism: before any `Write|Edit|MultiEdit` tool call, a hook script reads the `paths:` glob frontmatter of every `.claude/rules/*.md`, matches it against the path being edited, and emits the matching rule body to stdout. Claude Code injects that stdout as a `<system-reminder>` into the model’s context before the edit fires.

Why it’s the highest leverage: - Already-written rules become unmissable - Zero extra discipline required from agents - One hook covers all rule files automatically - Failures are silent (the edit still proceeds)

Pattern 2 – `Stop` correction-capture (medium leverage)

Closes: the gap where user corrections die in conversation instead of becoming permanent rules.

Mechanism: when the agent tries to stop, a hook reads the transcript, scans the user’s most recent message for explicit correction signals (regex on phrases like “that’s wrong”, “you forgot”, “I told you”, “stop doing”, “instead of ... use”, “you keep”, “you always”), and on match emits a `<system-reminder>` blocking the stop and asking the model to draft a one-line patch to the right `.claude/rules/<file>.md`.

Why it’s medium leverage: highly effective at hardening repeated lessons, but requires conservative regex to avoid false positives (firing on benign messages would erode trust quickly).

Pattern 3 – `Stop` build-gate (medium leverage, narrow scope)

Closes: the gap where commits land with a broken build because “run the build before commit” was procedural and skipped.

Mechanism: when the agent tries to stop, a hook checks `git status --porcelain` for changes to build-relevant files (TypeScript / Python / Go / etc., depending on stack). If any are dirty, it runs the build. On non-zero exit, it emits the build error tail and exits 2 (blocks stop). If the build passes or no relevant files are dirty, it exits 0.

Why it’s narrow: the build is slow (often 30+ seconds), so the hook MUST gate on whether build-relevant files are dirty. Otherwise it punishes documentation-only turns with full-stack builds.

Pattern 4 – `PostToolUse` `lint-fix` (low leverage, very low cost)

Closes: the “last 10% formatting drift” gap where small lint-fixable issues leak into CI and produce noise PR comments.

Mechanism: after every `Write|Edit|MultiEdit`, a hook runs the project’s auto-fix linter (e.g. `eslint --fix`, `ruff --fix`, `gofmt`) on the file just touched. **Always exits 0** – lint problems must NEVER block an edit; build correctness is enforced separately by Pattern 3.

Why it’s worth it despite low leverage: zero behavioural surface for the model, eliminates a class of CI noise. But also: skip if your project already runs auto-fix on save in IDEs or has a pre-commit hook outside Claude Code.

Pattern 5 – `Stop` `doc-freshness` gate (high leverage for teams that deploy often) – v1.2.0

Closes: the gap where a production push lands and the docs that describe production are not updated, so the next agent (fresh, no transcript) acts on a stale picture (Pitfall 21).

Mechanism: when the agent tries to stop, the hook walks the session transcript for two events – the last *production push* (a `git push` / `gh pr merge` whose own command segment references the protected branch) and the last edit to the project’s changelog. If a push happened and the changelog was not touched *after* it, the hook exits 2 with the full doc-update checklist on stderr. Loop-guarded with `stop_hook_active`; the agent can reply “that was not a production push” and stop.

Two guards are load-bearing, and both generalise to any transcript-scanning hook: a heredoc body is *data* (a doc that quotes `git push origin main` must not trip the gate), and a push is its own top-level command segment that *starts* with the verb – never a substring inside an `echo` or a comment.

Design rules for any hook

Rule 1 – Fail silent

A hook bug must NEVER block a tool call (or worse, lock the session). Wrap your script in try/catch around `main()`, exit 0 on any unexpected error, optionally write to stderr for debugging.

```
try { main(); } catch (e) {
  process.stderr.write(`[my-hook] ${e?.message ?? e}\n`);
  process.exit(0);
}
```

Rule 2 – Scope narrowly via path filters

A `PreToolUse` hook on `Write|Edit|MultiEdit` fires on **every** edit, including hook-owned scripts and `.md` files. Filter the file path early – exit 0 immediately if the path doesn’t match the hook’s purpose.

A build-gate hook should only run the build if files matching the build’s input glob are dirty. Otherwise the hook turns every documentation-only turn into a build wait.

Rule 3 – Cap output

Hook stdout becomes part of the model’s context. A 32 KB rule file dumped twice fills a meaningful slice of the context window. Cap each injection (e.g. 12 KB per rule, 40 KB total).

Rule 4 – Loop guards

Stop hooks that block (exit 2) get re-fired when the agent next tries to stop. Use `payload.stop_hook_active === true` as a “this hook already fired in this stop cycle, allow stop” signal. Without a loop guard you can pin the agent in a stop loop forever.

Rule 5 – Order matters in Stop arrays

If you have several Stop hooks (correction-capture + build-gate + doc-freshness), order them with the lighter / more-likely-to-fire one first. The first hook to exit 2 wins; the second never runs in that cycle. Putting build-gate (slow) first means correction-capture only runs on green builds. The shipped order is correction-capture → build-gate → doc-freshness.

Rule 6 – Stop hooks deliver via stderr, PreToolUse via stdout

Verified empirically against live Claude Code v2.x:

Hook event	stdout surfaces?	stderr surfaces?
PreToolUse	Yes – injected as additional context before the tool runs	yes
PostToolUse	(limited; not used for context injection in practice)	yes (for debug)
Stop	No – captured but not visible to the model	Yes – surfaced verbatim as <code>Stop hook feedback:\n[node ...]: <stderr></code> block in next turn

If you copy a PreToolUse pattern (write to stdout, exit 2) into a Stop hook, the model will NEVER see your reminder. We hit this exact bug shipping v1.1.1 of `correction-capture-prompt.mjs` – detection logic worked perfectly, exit code was 2, but the §9 reminder went to stdout and was discarded. The fix was a one-line change to `process.stderr.write(reminder)`.

The cost of getting this wrong is silent delivery failure, which is invisible to synthetic harnesses (the script returns the right exit code with the right content; the bug is in the harness contract, not the script). Only live integration testing in real Claude Code catches it.

When writing a new hook: - Stop hook reminder → stderr - PreToolUse hook context → stdout - Either hook’s debug logging → stderr (always works for visibility)

Rule 7 – Settings.json is loaded once at session start

Hook entries added to `.claude/settings.json` mid-session do **not** activate until a new session starts. This is the most surprising fact about hook deployment. After installing hooks: `/exit` and re-run `claude` for them to take effect. Verify in a brand-new session, not the one that installed them.

Rule 13 – Content beats ordering: accept a release entry that shipped inside the promote

Many teams write the changelog entry FIRST (changelog → integration branch → promote), so the entry travels inside the promote merge and no changelog edit follows the push in the transcript. An order-only gate fires on a correctly-freshened promote (it did, 2026-08-22). The doc-freshness gate therefore also accepts a same-day release heading in the changelog – read from `origin/<PROTECTED_BRANCH>` (the pushed truth, once fetched) or the working tree. Customise `RELEASE_HEADING_RE` to your changelog's heading shape; a heading for another environment or a stale date must not match.

Rule 8 – Team-shared hooks live in `.claude/settings.json`, not `.claude/settings.local.json`

`.local.json` is per-machine (gitignored). Hooks meant to enforce team rules belong in the shared `settings.json` so every contributor and CI run gets them. Leave `.local.json` for personal MCP allowlists.

Rule 9 – A killed check is inconclusive, not failed

A build-gate whose build is killed by the hook's own timeout has produced **no failure evidence**. Distinguish `status === null` (timeout / signal) from a non-zero exit; exit 0 silently on the former and let CI be the authority. A 5-minute cap on a 4-minute build, on a busy machine, produced an alarm loop every stop for a week (Pitfall 23). Size the cap to the slowest honest run.

Rule 10 – Quoted text is data: strip fences, inline code and heredoc bodies before matching

Every transcript-scanning hook (correction-capture, doc-freshness) must ignore markdown code fences, inline code spans and heredoc bodies before it pattern-matches. A test plan that *quotes* a correction phrase, or a doc that *shows* a `git push` command, is not the event itself. And match shell commands per top-level segment (`split` on `;`, `&&`, `||`, `|`, newline) with the verb anchored at the start – a `git push` buried inside an `echo` argument is not a push (Pitfall 22).

Rule 11 – Anchor correction regexes; a false positive costs more than a miss

`you already` matched *"you already have access"*. Anchor the verb to what follows (`you already (did|changed|broke|said)`), never bare. The framework deliberately leans slightly permissive on real corrections, but every false fire on a benign sentence erodes the team's trust in the whole hook layer. Tighten the same day.

Rule 12 – A push in another repository is not this repo’s push

A transcript-scanning gate sees every shell command the session ran, including `cd ../other-repo && git push origin main`. Releasing three sibling repos from scratchpad clones fired the doc-freshness gate twice in one session. Track where the shell is per command segment (`cd`, `subshell cd`, `git -C`, `gh --repo`) and count a push only when its effective directory is inside the hook’s own repo (`payload.cwd`); treat an unknowable directory (`cd $VAR`) as not ours. The template carries this.

Where to put the scripts

```
.claude/
  scripts/                                ← team-shared hook scripts (committed)
    surface-matching-rules.mjs
    correction-capture-prompt.mjs
    build-gate.mjs
    lint-fix.mjs
    doc-freshness-gate.mjs
  settings.json                          ← team-shared hook wiring (committed)
  rules/                                 ← unchanged
    *.md
```

Plus an orientation note at `docs/ai-context/HOOKS.md` explaining each hook’s purpose, behaviour, test command, and rollback.

Verification – how to know it’s working

You cannot verify hooks in the session that installed them (Rule 7). The verification recipe:

1. **Direct script tests** – run each script with synthetic stdin:

```
echo '{"tool_name":"Edit","tool_input":{"file_path":"src/foo.ts"}}' \
| node .claude/scripts/surface-matching-rules.mjs
```

If output is reasonable, the script logic works.

2. **Fresh-session integration test** – `/exit` and start a new Claude Code session. Edit a file matching a rule glob. Confirm the rule body appears as a `<system-reminder>` before the edit fires. If it doesn’t appear:

- Check `.claude/settings.json` is valid JSON (`node -e "JSON.parse(require('fs').readFileSync('.claude/settings.json'))"`)
- Check the script runs standalone without errors
- Check the path you’re editing actually matches some rule’s `paths`

3. **Negative test** – edit a file that should NOT trigger. Confirm no spurious output. Hooks that fire on every turn quickly become noise.

4. **Failure-mode test** – for the build-gate, deliberately introduce a compile/type error in your stack. Confirm the agent can't end the turn until the error is fixed.
5. **Loop-guard test** – for any `Stop` hook, introduce the trigger condition, fix it, ensure the second stop attempt succeeds (not blocked again).

If any of these fail in a real-world session, treat it as P0 – broken hooks are worse than no hooks because they erode trust.

When to remove hooks

Hooks have a lifecycle. Remove them when:

- The underlying problem they solved no longer exists (e.g. the recurring correction stopped happening for a quarter)
- They produce false positives more than 1× per week
- They've added > 30 seconds of latency to typical turns
- Maintaining them costs more than the bugs they prevent

Removal is one stanza-deletion in `.claude/settings.json`. Don't sentimentally keep dead hooks "just in case."

Templates

The framework ships generic templates for all five hook patterns at `templates/hooks/`:

Template	Purpose
<code>templates/hooks/surface-matching-rules.mjs.template</code>	Pattern 1 – generic rule-surfacing script
<code>templates/hooks/correction-capture-prompt.mjs.template</code>	Pattern 2 – generic correction-capture script
<code>templates/hooks/build-gate.mjs.template</code>	Pattern 3 – generic build-gate script (parameterized for stack)
<code>templates/hooks/lint-fix.mjs.template</code>	Pattern 4 – generic lint-fix script (parameterized for stack)
<code>templates/hooks/doc-freshness-gate.mjs.template</code>	Pattern 5 – production push ⇒ changelog touched in the same turn (v1.2.0)
<code>templates/hooks/settings.json.snippet</code>	Sample wiring for all five hooks
<code>templates/hooks/HOOKS.md.template</code>	Orientation note for <code>docs/ai-context/HOOKS.md</code>
<code>templates/slash-command.md.template</code>	Generic slash-command template (e.g. <code>/commit-push-pr</code>)

Each template has placeholders marked `<UPPERCASE_LIKE_THIS>` for stack-specific values (build script name, lint command, file globs, etc.).

Cross-links

- `docs/01-PRINCIPLES.md` § Principle 5 – runtime-enforced vs documented
- `docs/05-RULES-AND-SKILLS.md` – what rule files look like (the input to Pattern 1)
- `docs/08-COMMON-PITFALLS.md` § Pitfall 3 – why documentation isn't enforcement
- `docs/08-COMMON-PITFALLS.md` § Pitfall 17 – settings.json loaded once at session start
- `templates/hooks/` – drop-in scripts and configs
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` – the doc-freshness rule Pattern 5 enforces

11 – Project Truth, Learnings, and the Evidence Ladder

Added in v1.2.0. Distilled from three further months of running the framework on a production codebase with many parallel agent sessions. Everything here is tech-stack and domain agnostic.

The v1.0 framework answered “*who does the work and how do they talk to each other?*” (agents, handoff schema, rules). Three months of real use exposed the next failure class: **a fresh agent with no transcript knows only what is in git – and git mostly recorded code, not truth.**

Symptoms, all observed more than once:

- An agent built a feature that already existed, because nothing said “this exists, it is just hidden behind a flag”.
- An agent treated a staging-only feature as live in production, because the orientation map described the feature but not *where it was deployed*.
- A “we’ll do that later” spoken at the end of a session vanished, and was re-discovered as a bug two weeks on.
- A rule file documented the wrong HTTP verb for a route. Three agents trusted it. The fourth read the route.

The fix is three more documents, one playbook, and one enforced habit. This chapter specifies them.

The three knowledge stores (and what goes where)

Store	File	Answers	Written by	Loaded
Project truth	docs/ai-context/PROJECT.md	<i>What is true right now?</i> Environments, what is live where, sources of truth per domain, verified commands, definitions of done	Humans + agents, on every state change	Read at the start of every substantial task
Learnings	docs/ai-context/LEARNINGS.md	<i>What did we decide, what failed, what keeps going wrong, how has the owner corrected us?</i>	Agents, same turn a lesson emerges	Skim §D (corrections) before the first edit
Backlogs	docs/<AREA>_BACKLOG.md	<i>What did we defer, and when should we come back?</i>	Agents, before any turn that names deferred work ends	On “what’s pending on X?”

These sit alongside the stores v1.0 already had: `CLAUDE.md` (router), `.claude/rules/*.md` (path-scoped hard rules), `docs/ai-context/<area>.md` (orientation), `docs/<UPPERCASE>.md` (canonical reference), and the tool’s own auto-memory.

One home per fact. Every other place links to it. When the same rule turns up in two files, consolidate and leave a pointer. The single most corrosive doc smell is two copies that drift.

Where a new piece of knowledge goes

Kind of knowledge	Home
Universal behaviour + routing	CLAUDE.md (keep under ~200 lines)
Current state – what is live where, flags, applied migrations	PROJECT.md §3 + docs/CHANGELOG.md
A decision and its rationale	LEARNINGS.md §A
An approach that failed or proved unsafe	LEARNINGS.md §B
A recurring bug: symptom → first thing to check	LEARNINGS.md §C
A correction the owner had to give more than once	LEARNINGS.md §D and a rule file if path-scoped
A hard rule scoped to file paths	.claude/rules/<domain>.md
Per-area orientation (50-150 lines)	docs/ai-context/<area>.md
Deferred work	docs/<AREA>_BACKLOG.md
A multi-step repeatable procedure	.claude/skills/<name>/SKILL.md
Machine-private session pointers	Auto-memory – not visible to other machines or agents; anything durable must <i>also</i> land in one of the above

Don't persist: session-specific context, one-off data values, anything derivable from code or git in seconds, generic engineering advice.

PROJECT.md – the freshness contract

Template: templates/PROJECT.md.template. The sections that earn their keep:

1. **Product + roles** – one paragraph and a table of roles with their permission boundaries.
2. **Environments, branches, deploy** – and, critically, **what the deploy pipeline does NOT do** (migrations, cache purges, config pushes). “Migrations don't auto-apply” cost a real outage.
3. **Current state – what is live where.** A table: feature → prod / staging / flag / date. This is the single biggest fresh-agent trap: most recent work is usually staging-only and flag-gated.
4. **Repository map** – verified, date-stamped.
5. **Sources of truth by domain** – the canonical helper / table / module for each concept, so an agent reuses instead of re-implementing.
6. **Verified commands** – copied from the real build/test config, with the date.
7. **Definitions of done per change type.**
8. **Known sharp edges + unverified items.**

The contract: every fact is date-stamped, and the header names the commit it was verified against. If today is much later than the stamp, trust docs/CHANGELOG.md over the state table, *then fix the table*. A state claim without a date becomes a lie the day after it stops being true.

LEARNINGS.md – decisions, failures, corrections

Template: `templates/LEARNINGS.md.template`. Six sections:

- **A. Decision records** – what was decided, by whom, why, dated.
- **B. Failed or unsafe approaches** – so nobody re-tries them. Each entry: what was tried, what broke, what replaced it.
- **C. Recurring bug patterns** – symptom → first thing to check. The cheapest section to write and the one that saves the most hours.
- **D. Agent behaviour corrections** – written as “*Instead of: ... / Do: ...*” pairs. These came from real corrections; treat them as standing instructions.
- **E. Working with the project owner** – the owner’s preferred formats and approval gates. Examples that generalise: *multi-item status = a table, not prose; open decisions = one numbered list, resolved one at a time, never drop an item; approval is per-incident and current-turn, never carried forward from an earlier session.*
- **F. Documentation maintenance protocol** – the “where does knowledge go” table above, plus: date-stamp state claims; prune on contact (a verified-stale entry is fixed or deleted in the same turn, never left); unresolved conflicts get a $\triangle$ CONFLICT flag for the owner, never a silent pick.

Where a lesson is already a hard rule, `LEARNINGS.md` carries one line and a pointer – never a copy.

Backlogs – deferred work must be written, not spoken

If an end-of-turn summary names any “we’ll do this later”, follow-up PR, known gap, or deferred phase, the agent **must** append it to `docs/<AREA>_BACKLOG.md` before the turn ends. Template:

`templates/BACKLOG.md.template`. Each entry needs:

```
### <Short title>
- **Why.** (one sentence on user value or technical reason)
- **Effort.** (days/weeks)
- **Where.** (file paths or component names if known)
- **Revisit when.** (the condition or signal to look for)
```

When done, prepend `✓ DONE – PR #N (date)` to the title and leave the body in place for context.

Two corollaries learned the hard way:

- **A backlog card is a claim, not evidence.** “Done” on a card proves nothing about production; verify state in `PROJECT.md` §3 or the deployed system.
 - **Backlog ids collide across concurrent sessions.** Before taking the next `AREA-N` id (or the next migration number), check the remote *and* other open PRs. Two sessions took the same number on the same day, twice.
-

Every production push freshens the docs – same turn, no exceptions

The moment a change lands on the production branch, a future agent sees only what is in git. So a production push must be paired, **in the same turn**, with updates to the full doc set for what shipped:

1. `docs/CHANGELOG.md` – the mandatory anchor (a release entry).
2. The affected `docs/ai-context/<area>.md` orientation map(s).
3. Any affected `.claude/rules/*.md` hard rules.
4. `PROJECT.md` §3 if the production *state* changed (new live feature, flag flip, migration applied).

Documentation discipline alone skipped this under deadline pressure roughly one push in five. It is now a Stop hook – `templates/hooks/doc-freshness-gate.mjs.template`, Pattern 5 in `docs/10-HOOK-HARDENING.md` – which refuses to end a turn in which a production push happened and `docs/CHANGELOG.md` was not touched afterwards. Teams without hooks encode the same list in the orchestrator's Definition of Done and in the `/commit-push-pr`-style command.

The engineering playbook – six gates and the evidence ladder

Template: `templates/engineering-playbook-skill.md.template`. A project-wide skill that every substantial task runs through; the per-task-type skills (investigate-bug, build-feature) encode the checklists, this one encodes the gates they share. **If a gate conflicts with a hard rule, the rule wins.**

Gate	What it forces
1 Scope	Restate the user-visible outcome; classify (bug / extension / pivot / data correction / migration / ops / docs); name affected roles and blast radius; say what is out of scope
2 Evidence	Read the minimum routed context; audit what exists before building (already supported / supported-but-broken / partial / not supported / not needed); trace the real read/write path; tag every claim with a confidence class
3 Design challenge	Smallest viable change + one rejected alternative; find the existing pattern to mirror; state the failure condition; run the standing impact list (permissions, sensitive data, every client, accessibility, rollout compatibility, flag gating, money paths)
4 Implement	Narrow diffs; new user-visible behaviour lands flag-gated default-off; tests at the right level; code + migrations + scripts + docs in the same change
5 Verify	The evidence ladder below – a green build is evidence, not proof
6 Report	Files, rules read, checks run with actual results , risks and what was <i>not</i> verified (with its confidence class), docs updated, follow-ups written to a backlog

The evidence-confidence taxonomy

Mandatory in plans and reports. Nothing below `verified-*` may silently become a premise.

Class	Meaning
<code>verified-code</code> / <code>verified-schema</code> / <code>verified-test</code> / <code>verified-git</code>	You looked at the thing itself
<code>documented-unverified</code>	A doc says so – including <i>this project's own rule files</i> – and you didn't check
<code>historical</code>	True at a past date; may be stale
<code>unknown</code>	Needs confirmation

The proof ladder (in order of authority)

1. **The user-visible flow, exercised** – drive the real screen, capture the result. UI claims need UI proof, never code-reading. Hard-to-reach states are *created* (test accounts, seeds), not skipped.
2. **The data, landed** – query for the rows the change was supposed to produce. Proxy signals (“queued”, “in scope”, “deployed”, “nothing to do”) prove nothing.

3. **The third party, confirmed** – if an external service is involved, check its dashboard or API, not your code's return value.
4. **Regression edges** – adjacent tests, permission boundaries, empty/error/loading states, *both* sides of any two-sided flow.

Two rules that belong to the ladder:

- **Verify by reading the row, not by reasoning.** Two confident mechanisms were predicted for one write; both were wrong; a single query settled it in seconds.
 - **Never report a negative result from a reader you have not verified can see the whole set.** Default pagination caps (ORMs, REST layers, search APIs) silently truncate. A "none found" from a truncating reader is not evidence – check the reader's ceiling first.
-

Multi-client parity (when more than one client consumes one backend)

A second client is an unusually good audit of the first, and the source of a whole class of drift. These rules are generic to any web + mobile, or any two consumers of one API, and they are what makes Chapter 12's cross-repo contracts workable:

- **One client is the functional source of truth; the others may differ in presentation only.** What a person can see *and* do may not differ between clients. Layout, density, device affordances may. A deliberate divergence is written down with its reason.
 - **Same heading ⇒ same endpoint.** Before building a panel that mirrors another client's panel, open that client's component and find what it actually *fetches*. Two cards with the same title and the same chart fed from two different endpoints showed one user two different answers to one question. Record the panel → source pairing in the orientation map.
 - **One brain.** Business logic lives server-side; clients render. If a client needs derived data another client computes locally, extend the API rather than porting the computation.
 - **Changing a contract? Grep every consumer in the same turn.** Clients pin response shapes; a renamed field fails silently at runtime (fetch succeeds, screen is empty). Keep a **mirror registry** – a table of code deliberately duplicated across clients (palette, label mappers, status derivations) – and change both sides in one PR.
 - **Never ship a client that hard-depends on a brand-new server route.** Clients and servers deploy on different clocks, and an installed client can be older *or newer* than the server. Probe or degrade, and say so on screen when degraded.
-

Smaller lessons that earned a line

- **Search before building.** Grep for the feature first. An existing implementation was rebuilt because nothing advertised it – and a "hide until ready" guard had silently removed it for a week. Guards that hide features need a visible "hidden because" signal somewhere.
- **Diagnostic scaffolding is removed in the same change as the fix.** An on-screen error line that cracked a bug becomes a permanent leak of raw technical text if left in.

- **Two words for one thing ships bugs.** Ambiguous vocabulary (the same word meaning two things in two subsystems) caused a silent filter miss in production. Keep a glossary in `docs/ai-context/GLOSSARY.md` ; adding a fifth name for an existing concept is a defect, not style.
 - **Corrections harden into rules, never into “I’ll remember”.** Memory is machine-local and invisible to every other agent.
-

Cross-links

- `docs/10-HOOK-HARDENING.md` – Pattern 5 (doc-freshness gate) enforces the production-push rule.
- `docs/12-MULTI-REPO-WORKSPACES.md` – the parity rules above applied across repositories.
- `docs/08-COMMON-PITFALLS.md` – Pitfalls 18-26 are the incident reports behind this chapter.
- `templates/PROJECT.md.template` , `templates/LEARNINGS.md.template` ,
`templates/BACKLOG.md.template` , `templates/engineering-playbook-skill.md.template` .

12 – Multi-Repo Workspaces (web + mobile + microservices)

Added in v1.2.0. Every platform fact in this chapter was verified against the official Claude Code documentation on 2026-08-22 (`code.claude.com/docs/en/memory` , `/sub-agents` , `/large-codebases` , `/headless`). Re-verify before relying on a behaviour – the platform moves.

The question this chapter answers

“We have one repo per microservice, plus a web repo, a mobile repo, and an API gateway. Does orchestration need agents at every level, or can a top-level orchestrator understand all the services? Should we create a separate workspace repo that holds the agent files and gitignores the sub-projects?”

Short answer: **three layers, each optional, each building on the one below**. The per-repo framework you already have is layer 1 and stays mandatory. Shared specialists move into a plugin (layer 2) the moment two repos would otherwise carry copies. A workspace repo (layer 3) exists only when tasks *routinely* cross repos – and it holds the cross-repo orchestrator and the contract map, **not** the specialists. The teammate’s instinct (separate repo, gitignored clones, no CI) is right; the part that needs correcting is *what lives there*.

Layer 3	workspace repo	← cross-repo orchestrator, service map, contracts, sync script
Layer 2	shared plugin	← specialists / skills / hooks that every repo would otherwise copy
Layer 1	each repo (required)	← CLAUDE.md router, <code>.claude/agents</code> , <code>.claude/rules</code> , hooks – chapters 1–11

What the platform does when you launch from a parent directory

This table is the whole design constraint. Launch `claude` in `workspace/` , with each repo cloned as a subdirectory:

Thing	Loaded from the workspace root?	Loaded from a child repo?
CLAUDE.md	At launch	On demand – when Claude reads a file inside that child
.claude/rules/*.md	At launch (unscoped) / on match (<code>paths:</code>)	On demand, same as above
.claude/agents/	Yes (scanned recursively – <i>Subagents</i> doc)	No – unless the child is added with <code>--add-dir</code>
.claude/skills/	Yes	Yes – once Claude has touched a file in that child (<i>Monorepos and large repos</i> doc: “skills from every subdirectory Claude touches during the session”)
.claude/settings.json – hooks, permissions	Only from the start directory	Never – a child’s build-gate / doc-gate / deny rules do not fire in a workspace session
Auto-memory	Keyed to the workspace’s git repo	Not the child’s

Two consequences drive everything below:

1. **A child’s rules and CLAUDE.md follow its files into the workspace session** – the discipline layer survives. Good.
2. **A child’s enforcement layer (hooks, permission deny rules) does not.** If the child relies on a Stop hook to gate its build or its docs, a workspace session editing that child bypasses it. So any *write* to a child either runs inside the child’s own session, or the workspace re-implements the gate. We choose the former.

Also verified, and it changes a v1.0 claim: **subagents can now spawn subagents** (default depth three below the main conversation, `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` to change). A two-tier orchestrator → orchestrator → specialist tree is possible in one process. This chapter still prefers process-level delegation for writes, for reason 2 above – not because nesting is impossible.

Layer 1 – every repo keeps its own framework install

Nothing in this chapter removes the per-repo install. Each service, the web app and the mobile app keep their own `CLAUDE.md`, `.claude/agents/<repo>-orchestrator.md` + specialists, `.claude/rules/`, hooks and `docs/ai-context/`. Reasons:

- Most tasks are single-repo. A service team opening `claude` in their service must get the full experience with zero knowledge that a workspace exists.
- Hooks only fire from the start directory, so the gates *must* live with the code they gate.
- The child’s orchestrator is what the workspace delegates to. No per-repo orchestrator, nothing to delegate to.

A small repo (a 3-endpoint microservice) may need only an orchestrator plus two specialists. The framework's sizing table (`03-AGENTS-GUIDE.md`) applies per repo, not to the fleet.

Layer 2 – shared specialists become a plugin, not copies

Twelve services will share `backend-api` , `database` , `observability` , `release-devops` , `security-privacy` almost verbatim. Copying the files into twelve repos guarantees drift. Claude Code's own guidance for this is a **plugin** – a versioned bundle of agents, skills, hooks and commands, published in an internal marketplace and enabled per repo with `enabledPlugins` .

What goes in the plugin:

- The generic specialists (`backend-api` , `database` , ...) with the framework's handoff validation and return schema blocks baked in.
- The shared skills (`investigate-bug` , `build-feature` , `engineering-playbook` , `commit-push-pr`).
- The generic hooks (rule-surfacing, correction-capture, build-gate, doc-freshness-gate) with their stack parameters left for the repo to set.

What stays in each repo:

- `CLAUDE.md` , `.claude/rules/` (the hard-won, repo-specific gotchas), `docs/ai-context/` , `PROJECT.md` , `LEARNINGS.md` , the backlogs.
- The repo's orchestrator (it names *this* repo's specialists and docs).
- Any specialist unique to that repo.

Plugin agents are namespaced (`plugin-name:agent-name`), so they never collide with a repo's own agents – and a repo can still override by shipping a same-purpose local agent and listing only that one in its orchestrator's `Agent(...)` allowlist.

Skip this layer while you have fewer than three repos. Copy-and-diverge is fine at that size.

Layer 3 – the workspace repo

Create it when a task description regularly contains more than one repo name ("add the field to the orders service *and* show it on web *and* mobile"). Until then, the per-repo orchestrators plus a human carrying context between two terminals is cheaper than the layer.

Layout

`templates/workspace/bootstrap.sh.template` creates all of this from a filled `workspace.json` (it fills the per-repo placeholders, generates `.gitignore` and the `settings.json` deny block from the manifest's paths, fills the service-map rows, and lists what is left for you to fill by hand). The layout it produces:

```

workspace/                                ← its own git repo
├─ CLAUDE.md                             ← workspace router
(template/workspace/CLAUDE.md.template)
├─ workspace.json                         ← manifest: repos, contracts, commands
(template/workspace/workspace.json.template)
├─ .gitignore                             ← every child clone directory
├─ .claude/
│   └─ agents/
│       └─ <ws>-orchestrator.md           ← cross-repo coordinator
(template/workspace/orchestrator-agent.md.template)
│       └─ contract-guardian.md           ← REVIEW-ONLY: API/event/schema contract changes
across producers + consumers
│       └─ service-mapper.md              ← read-only: "which repo owns X", builds/refreshes
SERVICE_MAP.md
│       └─ rules/
│           └─ cross-repo-contracts.md     ← the contract-change protocol (unscoped: loads every
session)
│       └─ scripts/
│           └─ sync-repos.sh               ← clone/fetch every repo in the manifest
│           └─ delegate.sh                 ← run a child repo's orchestrator in the child's own
session
│       └─ return-schema.json              ← the return block as JSON Schema, for `claude -p --
json-schema`
│       └─ settings.json                  ← Bash allowlist for the two scripts + the deny block
on every child path (hooks optional)
├─ docs/ai-context/
│   └─ SERVICE_MAP.md                     ← repo → owns → produces → consumes → orchestrator
name
│   └─ CONTRACTS.md                       ← every cross-repo contract: producer, consumers,
spec location, versioning rule
│       └─ HANDOFF_SCHEMA.md               ← the standard schema + the two cross-repo fields
below
├─ web/                                  ← gitignored clone
├─ mobile/                               ← gitignored clone
├─ services/
│   └─ orders/                            ← gitignored clone
│   └─ billing/                           ← gitignored clone

```

Plain clones (not submodules) in gitignored directories, driven by `workspace.json` and `sync-repos.sh`. Submodules pin commits, which is exactly wrong for active development – you want each child on whatever branch its own team is on. The manifest is the source of truth for *which* repos belong; the clones are disposable.

No CI at the workspace level. Nothing deploys from here. The one optional job is a contract lint (run `contract-guardian` headlessly on a schedule and open an issue on drift).

The workspace orchestrator's contract

It is a coordinator, like every orchestrator in this framework – **no Edit/Write**. Its job:

1. Restate the cross-repo task. Read `SERVICE_MAP.md` and `CONTRACTS.md`. Identify which repos the task touches and which contracts sit between them.
2. **Order the work by the contract.** Producer first (additive change, deployed), then consumers. A consumer must never depend on a producer change that is not yet deployed – see “probe or

degrade" below.

3. Delegate **one handoff per repo**, each to that repo's own orchestrator, using the standard schema plus the two cross-repo fields. Sequence them; do not fan out writes to two repos that share a contract in parallel.
4. Use the workspace's own read-only specialists (`contract-guardian` , `service-mapper`) for anything that needs to see several repos at once.
5. Aggregate the returns. Refuse to declare the task done while any `contract_impact: breaking` handoff lacks a matching consumer handoff.

Two delegation mechanisms – pick by whether the child is written to

Mechanism A – delegate to the child's orchestrator in the child's own session (default for any write). `delegate.sh <repo> <handoff.yaml>` runs:

```
cd "<repo path from workspace.json>" && claude -p "$(cat handoff.yaml)" \
  --agent <repo>-orchestrator \
  --permission-mode acceptEdits \
  --allowedTools "Bash(git *),Bash(<build command> *),Bash(<test command> *)" \
  --max-turns 40 \
  --output-format json --json-schema "$(cat return-schema.json)"
```

Why this and not a nested subagent: `claude -p` started *in the child* loads the child's `CLAUDE.md` , rules, agents, skills **and runs the child's hooks** (it is documented to do so unless `--bare` is passed – never pass `--bare` here). The child's build-gate and doc-gate fire. The child's orchestrator spawns the child's specialists with the child's allowlists. Every layer-1 guarantee holds. `--json-schema` forces the `return:` block into the `structured_output` field, so the workspace orchestrator parses a contract, not prose; `session_id` in the same JSON lets it `--resume` that child session for a follow-up instead of starting cold.

Costs: a full child session per delegation (slower than a subagent), and you must pre-approve the tools the child needs because `-p` never prompts. Treat the allowlist as the child's permission budget and keep it narrow.

Mechanism B – workspace-level specialists that read across repos (default for anything read-only). `contract-guardian` and `service-mapper` run as ordinary subagents of the workspace orchestrator. They have file access to every child (the children are under the start directory), and each child's `CLAUDE.md` + rules load as the specialist reads its files. They are REVIEW-ONLY or read-mostly by contract and by `tools:` ; the `permissions.deny` block in the workspace `settings.json` additionally denies `Edit / Write` under every child path, so a workspace session *cannot* write to a child except through Mechanism A.

Mechanism C (optional) – pull a child's agents into the session with `--add-dir` . Documented for added directories; it makes the child's specialists spawnable from the workspace session and, with nesting, even a child orchestrator → child specialists tree inside one process. Two catches: every child ships a `backend-api` , so names collide (identity comes from the `name` field only), and the child's hooks still do not fire. Use it for fast cross-repo *investigation* when B is too coarse, never for writes. Confirm what actually loaded with `/agents` in your first session.

Two new handoff fields (additive – `schema_version` stays 1)

Outbound:

```
repo: <repo name from workspace.json – the ONLY repo this handoff may edit>
contract_impact:
  level: <none | additive | breaking>
  contracts: [<names from CONTRACTS.md>]
  consumers_to_update: [<repo names – required when level != none>]
```

Inbound:

```
contracts_changed:
- contract: <name>
  change: <one line – field added / field renamed / event payload changed ...>
  backward_compatible: <true | false>
  consumers_grepped: [<repo>:<path>, ...] # evidence the consumer search happened
```

A specialist that finds itself needing to edit a second repo returns `status: blocked` with `recommended_next_agent` naming that repo's orchestrator. It never reaches across.

The cross-repo contract protocol (`.claude/rules/cross-repo-contracts.md`)

These are the multi-client parity rules from Chapter 11, applied across repository boundaries:

1. **Producer first, additive only, deployed before any consumer depends on it.** A breaking change is two additive changes plus a removal after every consumer has moved.
2. **Grep every consumer in the same turn as the contract change**, and record the search in `consumers_grepped`. Clients pin shapes; a renamed field fails silently (request succeeds, screen empty).
3. **Never ship a consumer that hard-depends on a producer change that might not be deployed.** Probe or degrade, and say so on screen when degraded. Deploy clocks differ per repo, and an installed mobile client can be older or newer than the server.
4. **Same heading ⇒ same endpoint.** Before building a consumer panel that mirrors another consumer's panel, open that consumer's code and find what it actually fetches. Record the pairing in `CONTRACTS.md`.
5. **One brain.** Logic lives in the producer; consumers render. A consumer that needs derived data asks for an endpoint, never ports the computation.
6. **One session for a whole cross-repo change.** Hand the workspace orchestrator the producer edit and every consumer edit together, plan first, then delegate in order. Splitting a cross-repo change across days re-derives decisions per repo and is where drift starts.

workspace.json

```
{
  "name": "acme",
  "repos": [
    { "name": "web", "path": "web", "url": "git@.../web.git",
      "default_branch": "develop", "orchestrator": "web-orchestrator", "build": "npm
      run build" },
    { "name": "mobile", "path": "mobile", "url": "git@.../mobile.git",
      "default_branch": "develop", "orchestrator": "mobile-orchestrator", "build": "npm
      run typecheck" },
    { "name": "orders", "path": "services/orders", "url": "git@.../orders.git",
      "default_branch": "main", "orchestrator": "orders-orchestrator", "build": "make
      build" }
  ],
  "contracts": [
    { "name": "orders-api", "producer": "orders", "consumers": ["web", "mobile"], "spec":
      "services/orders/openapi.yaml", "versioning": "additive-only; breaking = new major
      path" }
  ]
}
```

`sync-repos.sh` clones what is missing and fetches what exists; it never checks out or resets a branch (each child's team owns that). `delegate.sh` reads `path`, `orchestrator` and `build` from here so the orchestrator prompt never hard-codes a path.

Scheduled workspace routines – the contract guardian gets a clock

Layer 3 as specified above is reactive: the contract guardian blocks a bad handoff *when a session runs one*. But agent-era defects concentrate at the seams between repos – a contract changed in the producer and never propagated is invisible to every per-repo session until something breaks. Seams are exactly where standing routines (Chapter 13) pay off first:

- **contract-drift-daily** – re-derive each contract's consumer list from the child clones and diff reality against `CONTRACTS.md` + `SERVICE_MAP.md`; open a workspace PR (or per-repo issues) on divergence. This is `contract-guardian-agent` run on a schedule instead of on demand – same charter, same REVIEW-ONLY posture, plus the routine output contract.
- **service-map-freshener** – re-verify `SERVICE_MAP.md` rows (routes, owners, deploy targets) against the clones; a stale row gets a dated PR, not silent tolerance (Chapter 11's freshness contract, applied cross-repo).
- **workspace janitor** – prune stale clones/worktrees and re-sync from the manifest (`sync-repos.sh`), so delegation never runs against a repo state nobody chose.

Routine writes follow the same delegation rule as everything else in this chapter: a routine that must *change* a child repo delegates to that child's own session so the child's hooks fire; the workspace-level routine itself stays read-only plus reports. Conventions, budgets and the catalog: `docs/13-STANDING-ROUTINES.md`.

What NOT to do

- **Don't put the specialists in the workspace repo.** They belong with the code they edit, or in the plugin. A workspace that holds `backend-api` has re-created the one-agent-for-everything problem one level up.
- **Don't write to a child from a workspace session.** Its hooks and deny rules are not loaded. Delegate (Mechanism A). The workspace `settings.json` deny block exists to make this a runtime fact, not a documented wish.
- **Don't use git submodules for the children.** Pinned SHAs fight active development. Manifest + gitignored clones.
- **Don't give the workspace a deploy pipeline.** Each repo deploys itself. The workspace has nothing to build.
- **Don't fan out writes to two repos that share a contract in parallel.** Producer first, then consumers, sequentially.
- **Don't pass `--bare` to a child delegation.** It skips the child's hooks, CLAUDE.md and agents – the exact things you delegated to get.
- **Don't let a `-p` delegation run against a repo you don't trust.** `-p` runs that repo's hooks and MCP servers with no trust prompt. Delegate only to your own repos.
- **Don't assume a child's agents are visible.** They are not, unless `--add-dir` ed. Check `/agents`.
- **Don't carry a production approval across repos.** Approval to push *orders* to production is not approval to push *web*. Per repo, per incident, current turn.

POC recipe (one afternoon)

1. Pick two repos that share one contract – a service and one of its consumers. Confirm each has a layer-1 install with a working `<repo>--orchestrator` (`claude --agent <repo>--orchestrator` on a single-repo bug). If not, do that first; the workspace cannot help a repo that has no orchestrator to delegate to.
2. Create the workspace repo from `templates/workspace/`. Fill `workspace.json` with the two repos and the one contract. Run `sync-repos.sh`.
3. Launch `claude --agent <ws>--orchestrator` in the workspace. Ask for something additive that crosses the contract: *"add `estimated_delivery` to the order response and show it on the web order page."*
4. Verify, in this order: the orchestrator read `CONTRACTS.md` and ordered producer → consumer; the producer delegation ran in the child (`session_id` in the JSON result, the child's hooks visible in its output); the consumer handoff carried `contract_impact.level: additive` and the return carried `consumers_grepped`; the workspace session itself wrote nothing under either child (`git status` in both).
5. Break it on purpose: ask for a *rename* of a field. The correct outcome is a refusal to proceed until the handoff is re-issued as add-then-remove with both consumers listed.
6. Only then decide on layer 2. Count how many agent files the two repos have in common; three or more identical files is the signal to package the plugin.

Cross-links

- [docs/02-ARCHITECTURE.md](#) § Variations – the monorepo and mobile+web notes now point here.
- [docs/04-HANDOFF-SCHEMA.md](#) – the base schema the two cross-repo fields extend.
- [docs/11-PROJECT-TRUTH-AND-LEARNINGS.md](#) § Multi-client parity – the origin of the contract protocol.
- [templates/workspace/](#) – every file in the layout above.
- Official references (verified 2026-08-22): *Set up Claude Code in a monorepo or large codebase*, *Subagents* → *Let subagents spawn their own subagents*, *Run Claude Code programmatically*, *Memory* → *Load from additional directories*.
- [docs/13-STANDING-ROUTINES.md](#) – the routine conventions the scheduled layer above relies on.

13 – Standing Routines (scheduled autonomy)

The question this chapter answers

Everything before this chapter runs when a person asks. The orchestrator decomposes a task (Chapter 2), hooks enforce discipline while it runs (Chapter 10), and the truth files brief the next session (Chapter 11) – but nothing happens between sessions. This chapter adds the third leg: **routines** – narrow agent jobs that run on a schedule, produce small reviewable pull requests, and get *tuned* instead of babysat.

The pattern is not speculative. The Claude Code team ran it publicly on their own repos in Aug 2026: a fleet of daily routines (crash fuzzing, duplicate-abstraction unification, dead-code removal, flaky-test root-causing) opened **388 PRs in a few weeks, 180 of which were merged** after automated review plus human review, with roughly 1 in 50 PRs being noise. Everything below restates that practice stack- and domain-agnostically, folded into this framework’s existing conventions.

Where routines sit

Leg	Runs when	Chapter
Interactive orchestration	a person asks	02, 03, 06
Mechanical enforcement	a session does something	10
Standing routines	the clock says so	this one

A routine is *not* a cron script with an LLM inside. It is a specialist (Chapter 3) with a written charter, an output contract, a review gate, and a tuning log – the same discipline the framework applies to interactive agents, applied to unattended ones. If a job needs no judgment (bump a version, rotate a log), use plain automation; a routine earns its cost only where the work needs reading, reasoning, or writing code.

The seven conventions

- 1. One routine, one narrow charter.** “Maintain the codebase” is not a charter. “Find statically unreachable code and remove it” is. A routine that needs the word “and” in its charter is usually two routines. The charter lives in a checked-in file (`templates/routine.md.template`) so it can be diffed, reviewed, and tuned like any other framework file.
- 2. Output is small, self-contained PRs – never direct pushes.** A routine’s writes reach the integration branch only through a PR (Pitfall 11 applies doubly to unattended writers). Small and self-contained is what makes ten routine PRs a five-minute review instead of an afternoon.
- 3. Every fix-PR carries a repro and a truth table.** “No evidence, no claim” (Principle 3) does not relax when nobody is watching – it tightens. A crash fix shows the reproduction; a behavior change shows the before/after truth table, verified end-to-end on the real system, not a mock. A routine PR without evidence is closed without debate, and that closure feeds convention 6.

4. All routine activity reports to one place. One channel, one issue thread, or one dated log file per fleet – not scattered notifications. The point is that a person can read one surface each morning and know what ran, what it produced, and what died. Routines that fail silently are worse than no routines (see Pitfall 28).

5. A routine never merges its own PR. The gate is: automated review first (cheapest tier on every PR; a stronger review tier for anything touching money, auth, data deletion, or another repo's contract), then a human merge. The human's cost stays low *because* of conventions 2 and 3.

6. Wrong output tunes the routine, not just the output. When a routine PR is wrong, fix the PR if it's worth fixing – but always patch the routine's charter/prompt so tomorrow's run doesn't repeat it, and date the change in the charter's tuning log. This is the correction-capture loop (Chapter 10, Pattern 2) applied to unattended work. Expect a real noise rate – the public reference fleet ran at ~1 in 50 – and budget it; a routine whose noise stays above its budget after two tuning passes gets retired, not tolerated.

7. Attempt caps and a verified retire path. Every routine carries a per-run budget (time, turns, or tokens), an attempt cap after which it parks instead of grinding, and – critically – a *checked* completion write. One production team's generator pipeline ran for 17 days without a single job completing: the "done" write violated a database constraint, the error return was never read, and a reaper kept resurrecting the same satisfied jobs forever. The lesson generalizes to any unattended loop: **a best-effort write whose failure changes control flow must have its error read**, and "the routine ran" must never be reported as "the routine worked" (Pitfall 12, unattended edition).

Model and effort tiering

Match the model to the routine, not the fleet. The public reference fleet ran mostly on the mid-size frontier model, reserving the largest for a few hard routines – and noted that a smaller model is workable if you "spend a bit more time auditing PRs and iterating on routines' prompts, then adding in checks and guardrails." That trade is general: **cheaper model ⇒ tighter output contract, more automated checks, more sampling in review**. Set effort/budget per routine, and re-tier quarterly as models change (Pitfall 18).

A starter catalog

Charters that have earned their keep, restated generically – pick two, not ten:

- **Dead-code remover** – remove statically unreachable code. For *suspected* (dynamically reachable?) dead code: add logging first, check the logs next run, remove only then. The two-step is the difference between a janitor and a hazard.
- **Duplicate-abstraction unifier** – find similar-yet-slightly-divergent helpers/components and unify them behind one, with a truth table showing call-site behavior is unchanged.
- **Flaky-test root-causer** – pick the week's flakiest test, find the *root cause* (order dependence, time, shared state), fix it or delete it with a written justification. Never auto-retry as the fix.

- **Doc-drift checker** – diff Tier-1/Tier-2 docs (Chapter 7) against reality: dead links, stale counts, commands that no longer exist. Companion to the librarian (Pitfall 14), running daily instead of quarterly.
- **Crash fuzzer** (repos that ship an app) – drive the real app, find crashes, root-cause, open a fix PR with the repro attached.
- **Contract-drift checker** (workspaces) – see Chapter 12 § Scheduled workspace routines.
- **Workspace janitor** – prune stale worktrees, merged branches, dead clone directories (Pitfall 24’s cleanup crew).
- **Hill-climber** – a routine wrapper around the hill-climb skill (`templates/hill-climb-skill.md.template`): one metric, one target, one bounded iteration per run.

Running them on Claude Code

Three mechanisms, in order of preference:

1. **Product-native schedulers** – Claude Code’s routines/scheduled-task surfaces (verified to exist 2026-08-24; the product is moving fast here, so re-verify capabilities rather than trusting this page). Least infrastructure, best observability.
2. **CI cron + headless c laude -p** (Mode 4, Chapter 6) – the portable floor that works on any scheduler. Note the verified behavior (2026-08-22): `c laude -p` **runs the working directory’s hooks**, so a routine executing in a child repo gets that repo’s enforcement for free – this is the same property Chapter 12 relies on for cross-repo delegation, and it is why routines should `cd` into the repo they operate on.
3. **An agent teammate in a chat channel** (e.g. Claude in Slack) – when the fleet’s reporting channel and its trigger surface should be the same place.

Give every mechanism a kill switch a human can flip without a deploy (an env var, a channel command, disabling the workflow) – you will want it during an incident.

What NOT to do

- Don’t let a routine merge, push to the integration branch, or touch production state directly – PRs only, gates always.
- Don’t launch a fleet before the first routine has run for two weeks and had its noise tuned down – routines compound, and so does their noise.
- Don’t give a routine a broad toolset “to be safe” – charter-narrow tools, same as any specialist (Principle 5); an unattended agent with wide write access is your largest blast radius.
- Don’t run a routine without a budget and an attempt cap – a runaway loop discovered on the monthly bill is the canonical failure (see also REFINEMENT check 11).
- Don’t report “ran” as “worked” – completion means the *verified* completion write landed.
- Don’t skip the schedule and run “when someone remembers” – that is not a routine, that is Chapter 6.

Cross-links

- `templates/routine.md.template` – the charter file every routine checks in.
- `templates/hill-climb-skill.md.template` – the metric-loop skill routines can wrap.
- `docs/06-INVOCATION-MODES.md` § Mode 6 – where routines sit among the invocation modes.
- `docs/10-HOOK-HARDENING.md` – Pattern 2 (correction-capture) is the tuning loop's interactive twin.
- `docs/11-PROJECT-TRUTH-AND-LEARNINGS.md` – routine tuning logs follow the LEARNINGS entry shape.
- `docs/12-MULTI-REPO-WORKSPACES.md` § Scheduled workspace routines – the contract guardian on a clock.
- `docs/08-COMMON-PITFALLS.md` – Pitfalls 27 (context weight) and 28 (unattended jobs need a verified retire path).